

Introduction to Graph Neural Networks for Machine Learning Engineers

James H. Tanis^{*1,2}, Chris Giannella¹, Adrian V. Mariano¹, and
Daoud Meerzaman²

¹The MITRE Corporation, USA

²National Cancer Institute, USA

Abstract

Graph neural networks are deep neural networks designed for graphs with attributes attached to nodes or edges. The number of research papers in the literature concerning these models is growing rapidly due to their impressive performance on a broad range of tasks. This survey introduces graph neural networks through the encoder-decoder framework and provides examples of decoders for a range of graph analytic tasks. It uses theory and numerous experiments on homogeneous graphs to illustrate the behavior of graph neural networks under different training sizes and degrees of graph complexity, with an emphasis on oversmoothing and oversquashing.

Keywords: Graphs, machine learning, graph neural networks, graph representation theory, node classification.

1 Introduction

Relationships within data are important for everyday tasks like internet search and road map navigation as well as for scientific research in fields like bioinformatics. Such relationships can be described using graphs with real vectors as attributes associated with the graph's nodes or edges; however, traditional machine learning models operate on arrays, so they cannot directly exploit the relationships. This report surveys Graph Neural Networks

^{*}Corresponding author: jhtanis22@gmail.com

(GNNs), which jointly learn from both edge and node feature information, and often produce more accurate models. These architectures have become popular due to their impressive performance on graph analysis tasks. Consequently, the number of research papers on GNNs is growing rapidly, and many surveys exist.

Some surveys discuss graph neural networks in the context of broad families such as graph networks, graph representation learning and geometric deep learning [12, 46, 27, 19, 117]. Other surveys categorize GNNs by abstracting their distinguishing properties into functional relationships [120, 141, 136, 27, 144]. Although useful for organizational purposes, generality and abstraction can be difficult to understand for those new to the field. Other surveys have a narrow focus, for example to discuss efforts to improve a specific weakness in GNN architectures [96], or to survey GNN work on a particular task, such as fake news detection or product recommendation [89, 40, 100]. While valuable for those interested in the task, they provide little background in GNNs and therefore assume the reader already has that knowledge.

For this reason, a concrete introduction to GNNs is missing, so we provide one here by combining theory with examples of their behavior in practice. We begin by introducing GNNs as encoder-decoder architectures. To provide perspective on the ways GNNs are used, we discuss common GNN applications along with examples of task-specific decoders for turning features into predictions. We think that studying a few important examples of GNNs well will help the reader develop a feeling for the subject that would be difficult to achieve otherwise. We therefore focus on three convolutional and attentional networks, GCN, GraphSAGE, and GATv2, which are commonly used both as benchmarks and as components in other GNN architectures. We conduct numerous experiments with these GNNs at two training sizes and on thirteen datasets of both high and low complexity. Transformer-based graph models are also an increasingly active direction, but they are outside the scope of this introductory treatment.

Our theoretical part also introduces five key challenges in the field—low homophily, label scarcity, oversmoothing, oversquashing, and explainability. In the experiments section, we focus on the first four, examining how they manifest in practice and how far they can be mitigated by simple means. Our experiments section has several goals:

- Compare benchmark GNNs with other graph models.
- Characterize how homophily, label scarcity, oversmoothing, and oversquashing affect node classification performance.
- Assess the extent to which these challenges can be mitigated by simple

hyperparameter and architectural tuning.

- Contrast link prediction and node classification, highlighting differences in how they respond to depth and design choices.

We hope these experiments, combined with the theoretical sections, will enable newcomers to use GNNs more effectively and to improve GNN performance on their own problems. We also hope that experts will gain new insights from our empirical analysis.

2 Common applications

Graph neural networks are suited to a variety of graph tasks.

1. Node classification

This task focuses on assigning labels to the nodes of a graph. In social networks, for example, node classification can be used to infer user roles or interests, or to predict whether a person belongs to a particular group [14, 2]. Beyond social networks, node classification is also used to categorize documents, videos, or webpages into different topics or classes [58, 87].

In cancer research, node classification is particularly impactful. Here, nodes may represent entities such as genes, proteins, regulatory elements, or clinical variables, while edges capture relationships like co-expression, signaling interactions, or the regulation of gene activity. By classifying these nodes, graph algorithms can help identify genes that drive cancer growth, reveal dysregulated pathways, distinguish cancer subtypes, and predict therapeutic vulnerabilities, thus advancing the goals of precision oncology [71, 135, 23, 62].

2. Link prediction

In most cases, link prediction is formulated as a binary classification problem: given two nodes, the task is to predict whether an edge between them should exist. In social networks, for example, link prediction is used to infer current or future relationships between individuals [85]. In recommender systems, it can be viewed as predicting edges in a bipartite graph between users and items, where an edge indicates that a user is likely to interact with or prefer a product [60, 119]. Link prediction also includes discovering missing relations in knowledge graphs, a task commonly referred to as knowledge graph completion [81, 7].

In bioinformatics, link prediction is often used to infer missing or previously unknown relationships between biological entities. For example, models can suggest new gene–gene or protein–protein interactions, identify possible control links between genes that act as switches and the genes they switch on or off [28], and uncover potential drug–target pairs [130, 99]. In cancer research, link prediction can help find important gene interactions that drive tumor growth [28], discover pairs of genes where blocking both is especially harmful to cancer cells but less damaging to healthy cells [112, 70], and suggest new connections between drugs and specific cancer types or diseases [1], potentially supporting biomarker discovery and the development of new therapies.

3. Community detection

Community detection algorithms group nodes into clusters based on some notion of similarity or connectivity that depends on the problem setting. Most classical methods are not machine-learning based [102, 124], although more recent approaches can be trained in an unsupervised or semi-supervised way [9, 51]. Applications include identifying social groups in social networks [111], entity resolution [104], fraud detection [75], text clustering (e.g., grouping Reddit posts by topic) [47], and graph visualization [115, 22].

4. Node regression and edge regression

Traffic prediction aims to forecast conditions such as speed and volume in the near future based on road sensors, supporting tasks like travel time estimation and route recommendation [126, 36]. In this setting, the road network is modeled as a graph with intersections as nodes and road segments as edges, and sensors can be treated as additional nodes attached to this network. Predicting continuous traffic variables at sensor locations is therefore a node regression problem. Less commonly, edge regression is used to predict weights on road segments that represent traffic flow or counts [97]. Other examples of node regression include predicting house prices and weather variables [59], or forecasting the amount of internet traffic to web pages [95].

5. Graph classification and graph regression

Traditionally, determining a molecule’s properties requires time-consuming and expensive laboratory experiments. Molecular property prediction is therefore a key task in materials science and drug discovery, where the goal is to estimate properties quickly and cheaply in order to guide

the design of new materials and therapeutics. Molecules can be naturally represented as graphs, with atoms as nodes and chemical bonds as edges, and GNNs—designed to operate directly on graphs—have quickly proven well suited to this problem [91, 39, 121, 42].

3 Introduction to encoder-decoder models

Throughout the paper, vectors such as x_i are column vectors, and matrices such as $X = (x_i^T)_{i \in \mathcal{N}}$ are composed of row vectors.

An *attributed graph*, $\mathbf{G}$, has a set of nodes, $\mathcal{N}$, as well as edges that define how the nodes relate to each other. We restrict our attention to undirected graphs, so the edges can be represented by a symmetric adjacency matrix, $A = (A_{ij})$ where $i, j \in \mathcal{N}$. An entry A_{ij} is one if an edge connects node i to node j and zero otherwise. Because the graph is *attributed*, each node, $i \in \mathcal{N}$, has an attribute, $x_i \in \mathbb{R}^\ell$ for some $\ell \in \mathbb{N}$. An attributed graph is thus defined by

$$\mathbf{G} = \left(\mathcal{N}, A, (x_i^T)_{i \in \mathcal{N}} \right). \quad (1)$$

Encoder-decoder models on graphs are a class of machine learning models. Machine learning on graphs presents challenges that do not arise in conventional machine learning on vectors, because graphs are irregular data structures and do not have a natural coordinate system. In particular, standard convolutional neural networks for image arrays do not work on graphs, because the k -hop neighborhoods may be different for every node. Nonetheless, a typical first step for machine learning on graphs is to obtain for every node, a low-dimensional feature vector that contains all the information needed to complete the desired task. These feature vectors are real vectors that often contain the information needed to represent the local edge structure about each node.

A feature vector of a node is also called a node embedding or a node representation, and collectively the feature vectors can be used for tasks on nodes, tasks on edges, or tasks on the entire graph. At the graph level, applying principal component analysis (PCA) or t-distributed stochastic neighbor embedding (t-SNE) to node embeddings can produce lower dimensional representations that enable visualizations to help understand how the algorithms are performing [108]. In addition, community detection algorithms use node embeddings to define the communities, either in an end-to-end fashion [106] or as part of a two-step process by applying the k -means

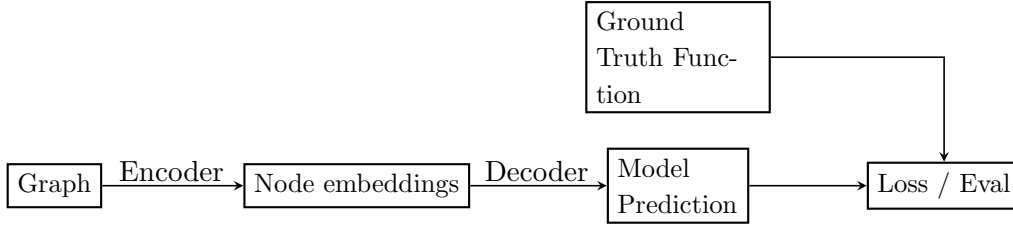


Figure 1

algorithm to the node embeddings [79, 31]. Node embeddings also support graph classification, where in the simplest case, a mean activation over all node embeddings of the graph determines the graph class. More sophisticated approaches are described in [88, 133]. Not surprisingly, node features are also used for node level tasks like node classification and regression [48, 58], as well as for edge level tasks like link prediction [132], edge classification [56, 57] or edge regression [68].

Due to the importance of node embeddings, there are many techniques to obtain them for a range of goals and data conditions. Perhaps the simplest example of a node embedding is given by the rows of an adjacency matrix. The map $i \rightarrow (A_{ij})_{j \in \mathcal{N}}$ defines node embeddings in $\mathbb{R}^{|\mathcal{N}|}$. These row vectors are poor feature vectors because they do not provide any structural information beyond each node’s 1-hop neighborhood nor do they account for any node attributes.

Instead, researchers may use rule-based descriptions of nodes, like centrality or clustering measurements, to produce low dimensional node representations that are more information dense, which may subsequently be applied to a downstream task with a traditional machine learning algorithm. The disadvantage of this approach is that the features are not part of the algorithm’s training process, so the features are not fine-tuned to minimize the loss function. To do this, researchers use an encoder-decoder approach.

3.1 Encoder-decoder framework

Many machine learning models adhere to an encoder-decoder framework shown in Figure 1. We restrict our attention to encoders that produce node embeddings because they are the most common and doing so makes the discussion more concrete.

Let $\mathcal{N}$ be a set of nodes, and let $\mathcal{G}$ be a set of attributed graphs on these

nodes. The *encoder* is a function,

$$\text{Enc} : \mathcal{G} \rightarrow \mathbb{R}^{|\mathcal{N}| \times \ell}, \quad \text{Enc}(\mathbf{G}) = (z_i^T)_{i \in \mathcal{N}}, \quad (2)$$

where $\mathbf{G}$ is a graph in $\mathcal{G}$. The encoder’s output is a matrix of node embeddings $z_i \in \mathbb{R}^\ell$, where $\ell \ll |\mathcal{N}|$. In practice, Enc is an algorithm whose computation involves the graph’s edges, node attributes, or both. A good encoder creates node embeddings that contain all of the information about each node that is required to complete the task at hand.

Subsequently the *decoder* function

$$\text{Dec} : \mathbb{R}^{n \times \ell} \rightarrow \mathbb{R}^k, \quad \text{Dec}(z_{i_1}, \dots, z_{i_n}) = \hat{y}, \quad (3)$$

converts n node embeddings into a prediction, $\hat{y}$, where k is the dimension of the prediction. We emphasize that the decoder generally does *not* simply invert the encoder. It is instead a kind of interpreter that “decodes” abstract node embeddings into predictions in order to solve the given task. Although decoders often include learnable parameters, they are usually simple functions whose parameters enter the computation in a relatively straightforward way, for example through a linear map or inner product. The majority of the model’s learnable parameters are usually in the encoder.

The *ground truth function*, Gt , of a graph, $\mathbf{G}$,

$$\text{Gt} : \mathcal{N}^n \rightarrow \mathbb{R}^k, \quad \text{Gt}(i_1, \dots, i_n) = y, \quad (4)$$

provides reference information that is known about the nodes of $\mathbf{G}$ in the form of k -dimensional vectors, where the integer n is the same as in the decoder function. For the node classification problem, $n = 1$, and the ground truth is the node’s class represented as a k -dimensional vector. The loss function compares the ground truth with the k -dimensional model predictions, and the training and evaluation algorithms use it to assess the quality of those model predictions.

There does not seem to be an accepted term in the literature that is appropriate in all contexts. Hamilton et al. [48] consider the case of a relationship between two nodes and call it a “pairwise similarity” function. “Pairwise similarity” is appropriate for link prediction, where the ground truth function may be a map $\mathcal{N} \times \mathcal{N} \rightarrow \{0, 1\}$ that says whether or not an edge exists between two nodes. However, “pairwise similarity” does not describe the case of node classification, where the map is from $\mathcal{N}$ to the set of node classes. In all cases, the role of the ground truth in the encoder-decoder framework is the same, so we use the same name: ground truth function.

The *loss* functions and *evaluation* metrics link the ground truth function and the model prediction. Most algorithms learn model parameters by some form of gradient decent. Common loss functions are cross entropy loss for classification, and L^1 or L^2 loss for regression tasks. Common evaluation metrics for classification include accuracy, average precision, F1, and AU-ROC (Area Under the receiver operator characteristic Curve); for regression tasks, common evaluation metrics are RMSE (Root Mean Square Error) and MAE (Mean Absolute Error).

3.2 Shallow embedding examples

We now present several representative examples of models that produce embedding lookups for nodes that were seen during the training process. These examples will illustrate the encoder-decoder framework and at the end we will note their shortcomings, which Hamilton et al. [48, 46] describe. This will lead us to more complicated encoder-decoder models called graph neural networks in the next section.

For each example, the input is a fixed matrix that provides a similarity statistic between any two nodes in $\mathcal{N}$ such as a weighted adjacency matrix. The output of these algorithms is a real vector (a feature vector) for each node describing the node’s neighborhood structure, and taken together, they support some downstream machine learning task.

The Laplacian eigenmaps algorithm is an early and successful nonlinear dimensionality reduction algorithm [13]. Given an attributed graph, $\mathbf{G}$, and a parameter $t > 0$, we can define a weighted adjacency matrix, W , for $\mathbf{G}$ by

$$W_{ij} = W_{ij}(t) = \begin{cases} \exp\left(-\frac{\|x_i - x_j\|^2}{t}\right) & \text{if } A_{ij} = 1, \\ 0 & \text{otherwise,} \end{cases} \quad (5)$$

where $i, j \in \mathcal{N}$, the vectors $x_i, x_j \in \mathbb{R}^\ell$ are the graph attributes, and A is the adjacency matrix as described in (1). Usually the Laplacian eigenmaps algorithm uses W as its input, but it can also use A or a k -nearest neighbor matrix.

The Laplacian eigenmaps algorithm can be reformulated in terms of the encoder-decoder framework [48, 46]. Define the ground truth, decoder and loss functions by

$$\text{Gt} : \mathcal{N} \times \mathcal{N} \rightarrow \mathbb{R}_{\geq 0}, \quad \text{Gt}(i, j) = W_{ij}, \quad (6)$$

$$\text{Dec} : \mathbb{R}^\ell \times \mathbb{R}^\ell \rightarrow \mathbb{R}^+, \quad \text{Dec}(u, v) = \|u - v\|^2, \quad (7)$$

$$L : \mathbb{R}_{\geq 0} \times \mathbb{R}^+ \rightarrow \mathbb{R}_{\geq 0}, \quad L(y, \hat{y}) = y\hat{y}. \quad (8)$$

Then the goal is to find the non-constant encoder

$$\text{Enc}(\mathbf{G}) = (z_i^T)_{i \in \mathcal{N}} \quad (9)$$

that minimizes the model's loss $\mathcal{L} \in \mathbb{R}^+$ up to a scaling factor, where that loss is

$$\begin{aligned} \mathcal{L} &= \sum_{i,j \in \mathcal{N}} L(\text{Gt}(i,j), \text{Dec}(z_i, z_j)) \\ &= \sum_{i,j \in \mathcal{N}} W_{ij} \|z_i - z_j\|^2, \end{aligned} \quad (10)$$

where the minimization is subject to a constraint that prevents the solution from collapsing to a lower dimension. Notice that $W_{ij} \geq 0$ is larger when i and j are adjacent, so (10) means that the model is punished during training for having node embeddings of adjacent nodes be far apart. In theory, the loss $\mathcal{L}$ could be zero, but in practice this only happens in idealized or degenerate situations. For example, a constant encoder gives the same output vector for every node, so $z_i = a$ for all $i \in \mathcal{N}$. This satisfies $\mathcal{L} = 0$ but is not useful.

Belkin et al. [13] provide an optimal solution based on generalized eigenvectors of the graph Laplacian. The graph Laplacian is the matrix $\Delta = D - W$, where D is a diagonal matrix defined by $D_{ii} = \sum_j W_{ji}$ [76]. The generalized eigenvectors $f_k \in \mathbb{R}^{|\mathcal{N}|}$, where $1 \leq k \leq |\mathcal{N}|$, are the solutions to the equation

$$\Delta f_k = \lambda D f_k, \quad (11)$$

where they are enumerated so that their corresponding eigenvalues are in ascending order:

$$0 = \lambda_0 \leq \lambda_1 \leq \dots \leq \lambda_{|\mathcal{N}|-1}. \quad (12)$$

The smallest eigenvalue, λ_0 , is always zero. The encoder's output for node i uses the ℓ eigenvectors corresponding to the smallest eigenvalues, excluding λ_0 :

$$z_i = [f_1(i), \dots, f_\ell(i)]^T. \quad (13)$$

These eigenvectors are used because, after excluding the trivial constant eigenvector, they are the graph's lowest-frequency nontrivial modes: they vary as smoothly as possible over edges with large weights, thereby preserving local neighborhood structure.

Inspired by the success of Laplacian eigenmaps, several papers define node similarity using inner products in a class of techniques called matrix factorization methods [3, 25, 82]. Matrix factorization approaches typically

rely entirely on the graph’s edge structure. The three cited methods differ only in how they define the ground truth function, Gt . In [48], Hamilton et al. reformulate each approach into an encoder-decoder framework:

$$\text{Gt} : \mathcal{N} \times \mathcal{N} \rightarrow \mathbb{R}^+, \quad \text{Method dependent} \quad (14)$$

$$\text{Dec} : \mathbb{R}^\ell \times \mathbb{R}^\ell \rightarrow \mathbb{R}, \quad \text{Dec}(u, v) = u^T v, \quad (15)$$

$$L : \mathbb{R}^+ \times \mathbb{R} \rightarrow \mathbb{R}^+, \quad L(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2. \quad (16)$$

Given an encoder Enc that provides the node embeddings, z_i ,

$$\text{Enc}(\mathbf{G}) = (z_i^T)_{i \in \mathcal{N}} \in \mathbb{R}^{|\mathcal{N}| \times \ell}, \quad (17)$$

the loss $\mathcal{L} \in \mathbb{R}^+$ is

$$\begin{aligned} \mathcal{L} &= \sum_{i,j \in \mathcal{N}} L(\text{Gt}(i, j), \text{Dec}(z_i, z_j)) \\ &= \frac{1}{2} \sum_{i,j \in \mathcal{N}} (z_i^T z_j - \text{Gt}(i, j))^2. \end{aligned} \quad (18)$$

If $Z = (z_i^T)$ is the matrix of features in $\mathbb{R}^{|\mathcal{N}| \times \ell}$, then the above loss satisfies

$$\mathcal{L} = \frac{1}{2} \|ZZ^T - S\|^2, \quad (19)$$

where S is the matrix with entries $S_{ij} = \text{Gt}(i, j)$. Minimizing $\mathcal{L}$ means finding a matrix Z that factors the ground truth of matrix S as shown in Equation (19), which is why the methods are called matrix factorization methods.

In [3], Ahmed et al. introduce the first matrix factorization method. They define the ground truth function by $\text{Gt}(i, j) = A_{ij}$, where A is the adjacency matrix. Hence, their goal is to find a solution that minimizes the loss

$$\mathcal{L} = \frac{1}{2} \sum_{i,j \in \mathcal{N}} (z_i^T z_j - A_{ij})^2. \quad (20)$$

More recently Perozzi et al. [87] introduce a method not obviously related to matrix factorization, where they use random walks on a graph as a tool to learn node embeddings that capture the edge structure of larger node neighborhoods in a computationally efficient manner. After a random initialization of node features, a stochastic gradient descent algorithm

updates features to optimize the information necessary to estimate the probability that two nodes co-occur on the same random walk of a fixed length. Grover et al. [45] improves on [87] by adding parameters that determine the algorithm’s tendency to explore new nodes and its tendency to return to the starting node. Finally [90] shows that these random walk methods are implicitly doing matrix factorization.

Matrix factorization methods have the benefit of working on graphs without attributes. On attributed graphs, however, all of the examples of shallow models share several shortcomings [48]:

1. They make insubstantial use of the node attributes during training, so they do not use all available information. Moreover, these models tend to define similarity in terms of proximity, and consequently they usually produce poor results when adjacent nodes in a graph tend to be dissimilar [93].
2. Trained models cannot be applied to unseen nodes without further training. This is impractical for dynamic graphs and for graphs that are so large that they cannot fit in memory. It also means that a model trained in a setting with a lot of labeled data is not transferrable to an unseen graph in a related domain with sparsely labeled data.
3. The information is not efficiently stored in the model. Each trained model is the collection of node features for the graph, which means model parameters are not shared across nodes. In particular, the number of parameters grows linearly with $|\mathcal{N}|$, which can create memory challenges for processing on large graphs.

The next section discusses more powerful encoder-decoder approaches called graph neural networks, which resolve these shortcomings.

4 Graph Neural Networks

Graph neural networks have several desirable properties. They jointly use node attributes and edge information for training, trained models can be applied to unseen graphs, and they apply naturally to both undirected and directed graphs.

This section focuses on GNNs that have so-called message-passing layers (described below). The vast majority of GNNs in the literature have message-passing layers.

4.1 Encoder layers

A typical GNN encoder has three classes of layers (see Figure 2): pre-processing layers, message-passing layers, and post-processing layers. The pre- and post-processing layers sometimes form the identity map.

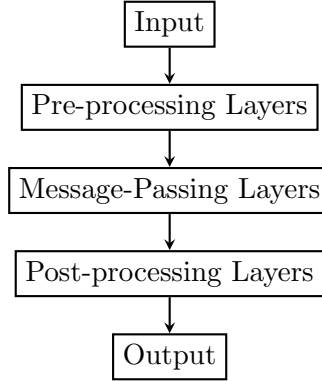


Figure 2: Three classes of layers in a GNN encoder.

The pre-processing layers map each node attribute vector x_i to a node feature vector $h_i^{(0)}$ in a computation that does not involve the edges of the graph:

$$\text{Pre-Proc} : \mathbb{R}^m \rightarrow \mathbb{R}^{m_0}, \quad \text{Pre-Proc}(x_i) = h_i^{(0)}. \quad (21)$$

Each layer in the pre-processing stack is a single-layer, fully-connected feed-forward neural network with trainable parameters.

The node features from (21) feed into the message-passing layers, which are the most important layers for the GNN’s performance [116]. While the pre-processing layers operate on each node independently, the message-passing layers operate on the entire graph, so we combine the individual node features $h_i^{(0)}$ into a feature matrix for the whole graph, $H^{(0)} \in \mathbb{R}^{|\mathcal{N}| \times m_0}$. Let K be the number of message passing layers. For $0 \leq k \leq K - 1$, the message-passing layer $(k + 1)$ is a map

$$\text{Message-Passing} \left((\mathcal{N}, A, H^{(k)}) \right) = H^{(k+1)} \quad (22)$$

from the graph with node feature matrix $H^{(k)}$ to the node feature matrix $H^{(k+1)} \in \mathbb{R}^{|\mathcal{N}| \times m_{k+1}}$. Message-passing layers produce their output node features by aggregating information from each node’s immediate neighbors in the graph. Then node features from each successive message-passing layer

contain information that has been aggregated over a wider set of nodes than the previous layers. The final output from message passing layers that aggregate node features over 1-hop neighborhoods is a set of low dimensional node features that summarize information in each node’s K -hop neighborhood. In this way, message-passing layers resemble the highly successful convolutional neural networks for image classification.

Node features from the message-passing layers subsequently feed into the final, post-processing layers of the GNN, which are, like the pre-processing layers, fully connected feedforward neural networks. Collectively the post-processing layers define the map

$$\text{Post-Proc} : \mathbb{R}^{m_K} \rightarrow \mathbb{R}^\ell, \quad \text{Post-Proc}(h_i^{(K)}) = z_i. \quad (23)$$

The combined action of all three groups of layers defines the encoder of the GNN, which produces a node embedding z_i for each node:

$$\text{Enc}(\mathbf{G}) = (z_i^T)_{i \in \mathcal{N}} \in \mathbb{R}^{|\mathcal{N}| \times \ell}. \quad (24)$$

Owing to the importance of the message passing layers, we describe them in greater detail next. Each message-passing layer computes its output using the same process. Let $h_i^{(k)} \in \mathbb{R}^{m_k}$ denote the output from layer k for node i . The output from message passing layer $k + 1$ is

$$h_i^{(k+1)} = \phi \left(\bigwedge \left(h_i^{(k)}, \bigoplus_{j \in \mathcal{N}_i} \mu_{ij} \right) \right). \quad (25)$$

The function, ϕ , sometimes called the update function, is differentiable and has trainable parameters. A common choice is a single layer, fully-connected, feedforward neural network. The $\bigoplus$ operation is a permutation-invariant aggregation function that combines several vectors into a single vector of the same dimension. Examples include element-wise sum, mean or maximum. The set $\mathcal{N}_i \subset \mathcal{N}$ is the 1-hop neighborhood of i (excluding i), and μ_{ij} is an expression that describes the interaction of node i with node j and depends on $h_i^{(k)}$ and $h_j^{(k)}$ and sometimes on the graph topology. The function $\bigwedge$ determines how each node interacts with the aggregation of its neighbors, for example by concatenation or by element-wise summation. Equation (25) is more concisely written without the function $\bigwedge$, but unlike [19], we include it because our experiments show the choice of $\bigwedge$ is at least as important to the behavior of the GNN as the expression μ_{ij} .

The literature describes three common architecture categories for GNNs: Convolutional, Message-Passing (MP), and Attentional. The type of a GNN

is determined by the choice of the expression, μ_{ij} . We denote the message-passing category by its initials, MP, to help distinguish it from message-passing layers. Our description of each GNN category follows [19]. This discussion is intended to capture key ideas rather than all subtle similarities and differences between individual models.

An architecture is in the convolutional category when (25) takes the form:

$$h_i^{(k+1)} = \phi \left(\bigwedge \left(h_i^{(k)}, \bigoplus_{j \in \mathcal{N}_i} \tau_{ij} \psi \left(h_j^{(k)} \right) \right) \right), \quad (26)$$

where ψ is a differentiable function that can have trainable parameters, usually an affine linear transformation, $\psi(v) = Cv + d$. The coefficients $\tau_{ij} \in \mathbb{R}$ are unlearned weights that usually depend only on the local graph topology and encode the connection strength between pairs of nodes [33, 58, 47, 116, 8]. If the graph exhibits homophily, meaning that nodes with similar features or the same class label tend to be neighbors in the graph [146], then in principal, the fixed weights τ_{ij} make these models a good choice. This occurs, for example, in a social network with users connected by friendship [4]. On the downside, the rigidity of fixed weights may inhibit their ability to represent the complex relationships that arise in low homophily graphs.

A GNN in the MP architecture category computes feature vectors by

$$h_i^{(k+1)} = \phi \left(\bigwedge \left(h_i^{(k)}, \bigoplus_{j \in \mathcal{N}_i} \psi \left(h_i^{(k)}, h_j^{(k)} \right) \right) \right), \quad (27)$$

where $\psi : \mathbb{R}^{2m_k} \rightarrow \mathbb{R}^{m_k}$ is a differentiable function with trainable parameters such as an affine linear transformation. These are the most expressive of the three flavors of GNNs, which makes them suitable for complex modeling tasks like predicting the properties of molecules or complex dynamical systems [11, 42]. However, this flexibility also makes them more challenging to train well compared to those in the convolutional category.

Finally, a model in the attentional category balances the expressiveness of an MP architecture with the simplicity of a convolutional one:

$$h_i^{(k+1)} = \phi \left(\bigwedge \left(h_i^{(k)}, \bigoplus_{j \in \mathcal{N}_i} a \left(h_i^{(k)}, h_j^{(k)} \right) \psi \left(h_j^{(k)} \right) \right) \right), \quad (28)$$

where a is a scalar-valued function that can have trainable parameters along with ψ [77, 131, 109, 18, 55]. For example, in [18] the function a is computed

by

$$a(h_i^{(k)}, h_j^{(k)}) = \frac{\exp(\alpha_{ij})}{\sum_{m \in \mathcal{N}_i} \exp(\alpha_{im})}, \quad \text{where } \alpha_{im} = v^T \cdot \sigma\left(W \cdot (h_i^{(k)} \parallel h_m^{(k)})\right). \quad (29)$$

Here, σ is a nonlinear function, $\parallel$ is concatenation, and for d hidden dimensions, $v \in \mathbb{R}^d$ is a vector and $W \in \mathbb{R}^{d \times 2m_k}$ is a matrix. Because a is scalar-valued, learning interactions involves fewer parameters than for the MP networks. This makes attentional networks easier to train and run than the MP networks at the cost of being less expressive. On the other hand, attentional networks tend to outperform convolutional ones on low homophily graphs [55, 140].

While the interaction between each node and the aggregation of its neighbors specified by $\bigwedge$ does not define the GNN category, it can significantly affect GNN behavior. When $\bigwedge$ is addition, like it is for GCN or GATv2, the sum mixes the node’s features with the aggregation of its neighbors. This tends to improve performance when neighboring node features are similar, and it tends to degrade performance otherwise. On the other hand, when $\bigwedge$ is concatenation, such as for GraphSAGE, the node’s features and the aggregation of its neighbors propagate separately, without mixing. In this case, $h_i^{(k)}$ and $\bigoplus_{j \in \mathcal{N}_i} \mu_{ij}$ in (25) are separate which tends to help when neighboring node features differ but hurt otherwise. See [47, 109, 18].

Table 1: Aggregation Function ($\bigwedge$) and GNN Category for Some Common GNNs

$\bigwedge$	GNN Category	
	Convolutional	Attentional
Sum	GCN	GAT, GATv2
Concatenation	GraphSAGE	

A version of convolutional GNNs also exist for the spectral domain, where an aggregation function operates on the eigenvectors of the graph Laplacian [20]. Comparing with the convolutional GNNs described above, the spectral version may provide richer features, but it also is more memory intensive and does not readily extend to directed graphs nor allow predictions on unseen nodes [134, 74].

4.2 Decoder and loss functions

Below are minimalist examples of these components for several graph tasks from Section 2. More sophisticated examples appear in the literature. As

usual, A is the graph's adjacency matrix and $\text{Enc}(\mathbf{G}) = (z_i^T)_{i \in \mathcal{N}}$ gives the node embeddings.

Given an integer $C > 0$, the softmax function, $\text{softmax} : \mathbb{R}^C \rightarrow \mathbb{R}^C$, is defined along each coordinate of a vector $v = [v_1, \dots, v_C]^T \in \mathbb{R}^C$ by

$$\text{softmax}(v)_j = \frac{\exp(v_j)}{\sum_{k=1}^C \exp(v_k)}. \quad (30)$$

Notice that $\sum_{j=1}^C \text{softmax}(v)_j = 1$.

1. Node classification

Let C be the number of class labels, and let $i \in \mathcal{N}$ be a node, and $y_i = [y_{i1}, \dots, y_{iC}]^T \in \{0, 1\}^C$ be a binary vector. If node i is in the c_{th} class, then $y_{ij} = 1$ if $j = c$, and $y_{ij} = 0$ otherwise. Then y_i is the class membership vector for i , which is ground truth. For a matrix $\Theta \in \mathbb{R}^{C \times \ell}$ with trainable parameters, the ground truth, decoder and loss functions are

$$\text{Gt} : \mathcal{N} \rightarrow \{0, 1\}^C, \quad \text{Gt}(i) = y_i, \quad (31)$$

$$\text{Dec} : \mathbb{R}^\ell \rightarrow (0, 1)^C, \quad \text{Dec}(z_i) = \text{softmax}(\Theta z_i), \quad (32)$$

$$L : \{0, 1\}^C \times (0, 1)^C \rightarrow \mathbb{R}^+, \quad L(y, \hat{y}) = -y^T \log(\hat{y}), \quad (33)$$

where $\log(\hat{y})$ is element-wise logarithm of the prediction vector $\hat{y} \in (0, 1)^C$, so L is the categorical cross-entropy loss function. Then the loss for the network is

$$\begin{aligned} \mathcal{L} &= \sum_i L(\text{Gt}(i), \text{Dec}(z_i)) \\ &= - \sum_i y_i^T \log(\text{softmax}(\Theta z_i)), \end{aligned} \quad (34)$$

where the sum is usually over a batch of nodes. See [58, 98, 116, 109].

2. Link prediction

The sigmoid function, $\text{sigmoid} : \mathbb{R} \rightarrow \mathbb{R}$, is defined for $t \in \mathbb{R}$ by

$$\text{sigmoid}(t) = \frac{1}{1 + e^{-t}}. \quad (35)$$

Define the decoder, ground truth and loss functions by

$$\text{Gt} : \mathcal{N} \times \mathcal{N} \rightarrow \{0, 1\}, \quad \text{Gt}(i, j) = A_{ij}, \quad (36)$$

$$\text{Dec} : \mathbb{R}^\ell \times \mathbb{R}^\ell \rightarrow (0, 1), \quad \text{Dec}(z_i, z_j) = \text{sigmoid}(z_i^T z_j), \quad (37)$$

$$L : \{0, 1\} \times (0, 1) \rightarrow \mathbb{R}^+, \quad L(y, \hat{y}) = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y}). \quad (38)$$

Then the loss is

$$\begin{aligned}
\mathcal{L} &= \sum_{(i,j)} L(\text{Gt}(i), \text{Dec}(z_i, z_j)) \\
&= - \sum_{(i,j)} A_{ij} \log(\text{sigmoid}(z_i^T z_j)) + (1 - A_{ij}) \log(1 - \text{sigmoid}(z_i^T z_j)),
\end{aligned} \tag{39}$$

where the sum is usually over a batch of node pairs. See [132, 65, 24] for more sophisticated examples.

3. Graph classification

Graph classification can be done like the node classification example but with one additional step. After the encoder produces the node embeddings, apply a global aggregator (e.g., entry-wise addition), which combines all node embeddings produced by the encoder into a single feature vector. This feature vector represents the graph and can be converted into a prediction, as done in the node classification example.

Specifically, consider a set of attributed graphs $\mathcal{G}$, and let $\mathbf{G} \in \mathcal{G}$ be a graph with nodes, $\mathcal{N}_{\mathbf{G}}$. Let the node embeddings of $\mathbf{G}$ be $z_i^{\mathbf{G}} \in \mathbb{R}^\ell$ for $i \in \mathcal{N}_{\mathbf{G}}$. In this example we aggregate them using addition so the aggregated feature vector for $\mathbf{G}$ is $z_{\mathbf{G}} = \sum_{i \in \mathcal{N}_{\mathbf{G}}} z_i^{\mathbf{G}} \in \mathbb{R}^\ell$. The rest follows like the node classification example. Let C be the number of class labels for $\mathcal{G}$, and let $y_{\mathbf{G}} = [y_{\mathbf{G}1}, \dots, y_{\mathbf{G}C}]^T \in \{0, 1\}^C$ be a binary vector. If $\mathbf{G}$ is in the c_{th} class, then $y_{\mathbf{G}j} = 1$ if $j = c$ and is zero otherwise. Then $y_{\mathbf{G}}$ is the class membership vector for $\mathbf{G}$, which is the ground truth. For a matrix $\Theta \in \mathbb{R}^{V \times \ell}$ with trainable parameters, the ground truth, decoder and loss functions are

$$\text{Gt} : \mathcal{G} \rightarrow \{0, 1\}^C, \quad \text{Gt}(\mathbf{G}) = y_{\mathbf{G}}, \tag{40}$$

$$\text{Dec} : \mathbb{R}^\ell \rightarrow (0, 1)^C, \quad \text{Dec}(z_{\mathbf{G}}) = \text{softmax}(\Theta z_{\mathbf{G}}), \tag{41}$$

$$L : \{0, 1\}^C \times [0, 1]^C \rightarrow \mathbb{R}^+, \quad L(y, \hat{y}) = -y^T \log(\hat{y}). \tag{42}$$

Then as in the node classification example, L is the cross-entropy loss function. The loss for the network is

$$\begin{aligned}
\mathcal{L} &= - \sum_{\mathbf{G}} L(\text{Gt}(\mathbf{G}), \text{Dec}(z_{\mathbf{G}})) \\
&= - \sum_{\mathbf{G}} y_{\mathbf{G}}^T \log(\text{softmax}(\Theta z_{\mathbf{G}})),
\end{aligned} \tag{43}$$

where the sum is over a batch of graphs selected from $\mathcal{G}$ for model training. Global mean and max aggregators are also possible. See [121, 88, 103, 114].

4. Community detection

Modularity is one of the most commonly used graph clustering metrics in the literature [38, 78]. Suppose we are trying to partition the graph's nodes into $C > 1$ clusters. Define

$$\delta(i, j) = \begin{cases} 1 & \text{if } i \text{ and } j \text{ belong to the same cluster,} \\ 0 & \text{otherwise.} \end{cases} \quad (44)$$

Let $|\mathcal{E}|$ denote the number of edges of the graph, and let d_i be the degree of node i . Then the modularity metric is

$$\mathcal{Q} = \frac{1}{2|\mathcal{E}|} \sum_{i,j \in \mathcal{N}} \left(A_{ij} - \frac{d_i d_j}{2|\mathcal{E}|} \right) \delta(i, j). \quad (45)$$

Hence, $\mathcal{Q}$ measures the divergence of the number of edges within clusters from what one would expect by random chance, that is, if the graph had the same node degrees but its edges were assigned by a uniform distribution. See [107] for the construction of random graphs with given node degrees, a method called the configuration model. Note that $\mathcal{Q}$ can be positive or negative, and a large positive value indicates an unusual number of intra-cluster edges, thereby indicating meaningful community structure.

Tsitsulin et al. [106] use the modularity metric to define the loss function for their GNN. First, they re-write $\mathcal{Q}$ in a convenient form for a gradient descent optimization. Let $d = (d_i)_{i \in \mathcal{N}}$ be the node degree vector, and let

$$B = A - \frac{dd^T}{2|\mathcal{E}|}. \quad (46)$$

Then

$$\mathcal{Q} = \frac{1}{2|\mathcal{E}|} \text{Tr}(U^T B U), \quad (47)$$

where $U \in \{0, 1\}^{|\mathcal{N}| \times C}$ is the cluster assignment matrix: $U_{ic} = 1$ if node i belongs to cluster c , and $U_{ic} = 0$ otherwise.

Next, relax the entries of U by allowing them to take values in the interval $[0, 1]$. This way we can apply continuous optimization methods

to $\mathcal{Q}$, which is differentiable with respect to the entries of U . Specifically, let $\Theta \in \mathbb{R}^{C \times \ell}$ be a learnable parameter matrix. Define the decoder by:

$$\text{Dec} : \mathbb{R}^\ell \rightarrow [0, 1]^C, \quad \text{Dec}(z_i) = \text{softmax}(\Theta z_i), \quad (48)$$

where row i of U is

$$U_i = \text{Dec}(z_i) \in [0, 1]^C. \quad (49)$$

Next we define the Gt function. This is somewhat of a misnomer for community detection problems because it lacks ground truth. However, the Gt function here serves the same purpose: guiding training. Define

$$\text{Gt} : \mathcal{N} \times \mathcal{N} \rightarrow \mathbb{R}, \quad \text{Gt}(i, j) = B_{ij} = A_{ij} - \frac{d_i d_j}{2|\mathcal{E}|}. \quad (50)$$

Then the decoder outputs probability estimates that a node is in a given cluster, which determine meaningful communities when the loss

$$\mathcal{L} = -\frac{1}{2|\mathcal{E}|} \text{Tr}(U^T B U) \quad (51)$$

has a large negative value. Notice that $\mathcal{L}$ is differentiable, so the graph neural network can be trained in an end-to-end fashion. See [106] for implementation details, which include a regularity term not included here.

We just discussed an unsupervised approach to community detection with graph neural networks, but there is also semi-supervised community detection. Here, the modeler incorporates knowledge that some nodes must be in the same class (must-link constraints) and some nodes cannot be in the same class (cannot-link constraints) [92]. Supervised community detection also exists, but this is not common and may be regarded as a type of node classification problem [30].

5. Temporal node regression

Our final example illustrates that problems with a time variable can fit within the same framework. A common node regression problem is to predict numerical values of traffic speed and volume at sensors located on a road network. These models can be complex, but a relatively

simple one appears in [113]. For each time $t \in \mathbb{N}$, their ground truth, decoder and loss functions are

$$\text{Gt}_t : \mathcal{N} \rightarrow \mathbb{R}, \quad \text{Gt}_t(i) \in \mathbb{R}, \quad (52)$$

$$\text{Dec}_t : \mathbb{R}^\ell \rightarrow \mathbb{R}, \quad \text{Dec}_t(z_i) = \sigma(\Theta_t^T z_i + b_t), \quad (53)$$

$$L : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}^+, \quad L(y, \hat{y}) = (y - \hat{y})^2, \quad (54)$$

where σ is a nonlinear function, $\Theta_t \in \mathbb{R}^\ell$ is a vector, and b is a scalar, both trainable. The goal is to predict the next $T_{\max} > 0$ time steps into the future, so their loss includes an average over those time steps. Specifically, their loss is the mean square error loss given by

$$\begin{aligned} \mathcal{L} &= \frac{1}{T_{\max} |\mathcal{N}_s|} \sum_{t=1}^{T_{\max}} \sum_{i \in \mathcal{N}_s} L(\text{Gt}_t(i), \text{Dec}_t(z_i)) \\ &= \frac{1}{T_{\max} |\mathcal{N}_s|} \sum_{t=1}^{T_{\max}} \sum_{i \in \mathcal{N}_s} \left(\text{Gt}_t(i) - \sigma(\Theta_t^T z_i + b_t) \right)^2, \end{aligned} \quad (55)$$

where $\mathcal{N}_s$ is the set of nodes with sensors. A loss function defined by mean absolute error is used by the top performing models [139, 69].

4.3 Learning paradigms

Inductive and transductive learning are the two common learning paradigms for graph neural networks. In inductive learning, *no test data* is available during training, whereas in transductive learning, *all test data except the test labels* are available during training. Thus, inductive learning for node classification is the usual supervised-learning setting. For example, consider a coauthor network, where each node is an author, an edge indicates that two authors have worked together, node features represent keywords from their papers, and the node label is the author's most active field of study [101]. In inductive learning, we may have a training graph that covers the years 2000–2004 and a separate test graph that covers 2005–2007. The test graph is not available during training. After training, the model is evaluated by comparing its predictions on the test graph with the known labels, i.e., the authors' most active fields of study. In transductive learning, the graph from 2000–2007 is available during training, but the labels of the test nodes, i.e., their most active fields of study, are withheld from training.

4.4 Challenges

4.4.1 Low Homophily

Homophily is the tendency for connected nodes to have similar characteristics, labels or features and is one aspect of graph complexity. There are different ways to measure homophily, such as feature homophily, which concerns similarity across features, and edge homophily, which is the fraction of the edges that connect two nodes of the same class [146].

For node classification problems, adjacent nodes in high homophily graphs tend to have the same labels. Message passing works well on these graphs because it aggregates similar classes, which tend to have similar features. This amplifies feature patterns and reduces noise. On the other hand, in low homophily graphs, adjacent nodes typically have different labels and node features, so neighborhood aggregation in message passing tends to mix classes together, which blurs class boundaries in feature space, leading to worse predictions.

For surveys considering GNNs on low homophily graphs, see [54, 44]. One approach for dealing with low homophily is to not treat all the neighbors equally, but instead skip or select neighbors that will be most useful, [145, 146, 86]. Node positional information can help attentional networks weight nodes appropriately while aggregating neighborhoods in a message passing layer, [73]. Other methods group feature vectors of node neighborhoods by similar characteristics, and then aggregate the groups of similar vectors separately, [53, 32].

4.4.2 Label Scarcity for Node Classification

Label scarcity in graphs is common in real applications due to the high cost of manual labeling, data privacy constraints, and large data set size, where graphs may contain billions of nodes. Training models on label scarce graphs tends to create models that do not generalize well. Additionally, the weaker signal in label scarce graphs tends to make model training less efficient.

Many techniques to address label scarcity assume that adjacent nodes on the graph have similar characteristics, sometimes called the homophily assumption. In fact, message passing is a smoothing operation that propagates similar information between neighbors, so under the homophily assumption, message passing networks are a good tool for mitigating label scarcity. Sometimes, further smoothing can improve node classification predictions by combining message passing networks with traditional label propagation, which is an iterative procedure for assigning labels to unlabeled nodes according

to the prevalence of labels in a given node’s neighborhood [147]. So-called heterophily-aware GNNs do not rely on the homophily assumption. Instead, they emphasize structural relationships of node neighborhoods and so reduce the importance of labels [145, 146]. Other techniques bypass the homophily assumption using a two-step process: first train an unsupervised model to produce generally informative features and then train a classifier on those feature vectors [110, 129].

4.4.3 Oversmoothing

Repeated neighborhood aggregations of node features tends to make feature vectors of nodes similar to those of their neighbors. At some point, nodes become harder to distinguish and predictions become worse, which is over-smoothing. For this reason, GNNs tend to have few layers unless there is some architectural modification from what we described in Section 4.1. One such modification is to add skip connections, which preserve information from previous layers by adding or concatenating the earlier feature vectors with those at later layers [29, 64]. This tends to stabilize the network by allowing each layer to learn a small refinement of previous layers rather than overwriting previous representations. Alternatively, several strategies use normalization layers within the network to preserve node feature diversity and prevent too much collapse [138, 143, 142]. Another common approach is Jumping Knowledge, which aggregates node features across layers in a way that allows the model to choose an appropriate neighborhood scale (i.e. receptive field) to combine node feature information [122]. Surveys on oversmoothing include [96] and [52].

4.4.4 Oversquashing

Unlike oversmoothing which is mostly a depth effect, oversquashing is mainly a graph topology effect. Intuitively, over-squashing occurs when too many signals need to squeeze through too few edges and a bottleneck occurs, which makes nodes insensitive to other far away nodes, [6].

Several approaches address over-squashing [96, 5]. A simple approach is to increase the capacity of information flow between nodes by increasing the number of hidden dimensions of the GNN [37]. Another simple approach is to use attention networks, like GATv2, that can prioritize certain channels and thus allocate more capacity to some channels than others. More sophisticated methods locate bottlenecks using negative edge curvature, which highlights edges between otherwise more weakly connected or distinct re-

gions, and effective resistance, which is large between node pairs that are connected by only a small number of alternative paths [105, 80, 15]. Still other approaches try to bypass bottlenecks using global attention mechanisms that enable nodes to learn from distant ones directly [118]. See [5] for a survey of oversquashing.

4.4.5 Explainability

For tasks where mistakes can be costly, user trust in GNN inferences is crucial, which in turn requires explanations that are understandable and aligned with how the model actually reasons. In the last ten years, many approaches have been developed for providing explanations in terms of graph structures or feature statistics. Local methods such as GNNExplainer [127] and CF-GNNExplainer [72] focus on individual inferences. GNNExplainer identifies a compact subgraph and feature subset that are most informative for an inferred node or edge label. CF-GNNExplainer takes a counterfactual approach and identifies the minimum update to the edge structure necessary to change an inferred node label. In effect the edge changes provide insight into node links crucial to the inference. Other approaches focus on providing explanations for more global GNN behavior. ProtGNN [137], unlike earlier methods, is applied during GNN training and modifies the underlying GNN architecture. Prototype vectors are produced for each label class allowing the inference of labels to be interpreted in terms of Euclidean proximity between the prototypes and the underlying node representations. GCfExplainer [61] addresses graph classification and aims to find a small set of graphs that can serve as close counterfactuals to many of the input graphs. In effect this small set of graphs illustrates the graph structures pertinent to a larger class of inferences. For a detailed discussion of approaches that provide explanations in terms of graph structures or feature statistics, see surveys [21] and [67].

More recently, approaches based on Large Language Models (LLMs) have been developed for providing textual explanations for GNN inferences. GraphXAIN [26] addresses node classification and aims to produce coherent narratives describing why a GNN produces inferred node labels. An example output for a “high salary” label on a node in a graph of basketball players is shown below (taken from Fig 1. in [26]).

”The Graph Neural Network classified NBA player represented by the target node with index 57 as having a ‘High Salary’. Central to this prediction is the player’s position as a Small Forward, which was among the most significant features. Players in this position, known for their

versatile roles on the court, often command higher salaries, which could contribute to the 'High Salary' prediction. The number of games played by the player also heavily influenced the model, as appearing in more games often signals competence and reliability, traits associated with higher earnings. Additionally, the player's height at 200.6 cm placed them around the middle of the height distribution among players, potentially aiding in their adaptability across positions, thereby making them a valuable team asset."

A similar problem was addressed in [83] though a different approach to using an LLM was developed. Other approaches have focused on producing textual, counterfactual explanations [50] and [43].

5 Experiments

This section complements the previous theoretical sections with experimental results on node classification and link prediction in the transduction setting (see Section 4.3 for a definition). The goal is to describe the behavior of GNNs under several training and dataset conditions. Our experiments focus on GCN, GATv2, and GraphSAGE because they are commonly used as benchmarks and many GNN architectures are built on top of them [84, 110, 63, 132, 106, 123]. Table 1 on page 15 summarizes characteristics of these GNNs. Our experiments include two other graph models: Multilayer Perceptron (MLP), which only uses node features, and DeepWalk, which only uses edges.

We use thirteen open-source datasets: seven high homophily datasets and six low homophily ones. In Section 4.4.1 we noted that there are different ways to measure homophily. In what follows, we mean *edge homophily*, i.e., the fraction of edges that connect two nodes of the same class. The high homophily datasets are citation networks (Cora, PubMed, CiteSeer, DBPL [125, 16]), co-purchase networks (AmazonComputers, AmazonPhoto [101]), and a coauthor network (CoauthorCS [101]). The low homophily datasets are webpage-webpage networks (WikipediaSquirrel, WikipediaChameleon, WikipediaCrocodile, Cornell, Wisconsin [95, 86]) and a co-occurrence network (Actor [86]). All datasets are homogeneous graphs, which means they have a single node type (e.g., "article") and a single edge type (e.g., "is cited by"). Ten of the datasets are node classification networks. Three of the datasets—Squirrel, Chameleon and Crocodile—are node regression datasets, so we transform them into node classification networks by partitioning their values into five ranges, where each range defines a class.

Table 2: Homophily and SNR values of datasets

Dataset	Num Nodes	Homo- phily	SNR	Dataset	Num Nodes	Homo- phily	SNR
Photo	7650	0.83	42.24	Crocodile	11631	0.25	327.69
DBLP	17716	0.83	13.94	Chameleon	2277	0.24	1,624.13
Cora	2708	0.81	71.09	Squirrel	5201	0.22	17.78
CoauthorCS	18333	0.81	260.26	Actor	7600	0.22	3.24
PubMed	19717	0.80	12.42	Wisconsin	251	0.20	81,019.76
Computers	13752	0.78	44.14	Cornell	183	0.13	103,016.58
CiteSeer	3327	0.74	967.96				

Homophily and the signal-to-noise ratio (SNR) of node features are two measures of complexity in an attributed graph. We use a Fisher-type version of SNR to measure the separation of node features when they are grouped by node classes. Suppose there are C node classes, and for each class $c \in \{1, \dots, C\}$, and let Σ_w be the pooled within-class covariance. To keep Σ_w well-conditioned, we use a shrunk covariance $\Sigma_w^{(\alpha)}$, for small α , and define SNR by

$$\text{SNR} = \frac{1}{C^2} \sum_{1 \leq b, c \leq C} (\mu_b - \mu_c)^\top (\Sigma_w^{(\alpha)})^{-1} (\mu_b - \mu_c), \quad (56)$$

where μ_{c_k} is the within-class centroid of the node features of nodes in class c_k . See Supplementary Material, Section S.1.1 for details. Table 2 reports these metrics for each dataset.

Unlike computer vision or language models, GNNs are often trained from random initialization rather than fine-tuned from a pre-trained model, because they are relatively small. We take the same approach. Using PyTorch Geometric, we run all experiments 25 times. Plotted curves in Section 5 show mean values over these runs and are averaged across collections of high- or low-homophily datasets; they are intended to summarize qualitative trends. For dataset-level node-classification results, 95% confidence intervals are reported in the Supplementary Material.

5.1 Baseline node classification performance

The baseline GCN, GATv2 and GraphSAGE architectures are two layer message-passing networks with 16 hidden dimensions and no pre-processing or post-processing layers. DeepWalk also has 16 hidden dimensions, and MLP is a three-layer fully connected network where the first layer has 128 hidden dimensions and the second one has 64. In the literature, GAT is a

Table 3: Average CPU wall-clock time in minutes on the Cora dataset (training + validation / test inference)

	GCN	GraphSAGE	GATv2	MLP	DeepWalk
Minutes	2.26 ± 0.26	2.23 ± 0.28	4.03 ± 0.23	2.58 ± 0.31	57.96 ± 4.29

benchmark more often than GATv2, but we use GATv2 because it consistently outperforms GAT in our experiments [18]. We trained the models for at most 200 epochs, with a learning rate of 0.1 and a train/val/test split. We used two training sizes, 80% or 1%, and we divided the labels not used for training evenly among the validation and test sets.

Table 3 reports end-to-end CPU wall-clock time Cora dataset [125], a dataset often used for benchmarking, including training and validation/test inference. We do not separate these here because the table is intended only as a coarse comparison across model families on a common CPU setting. The graph convolutional models have comparable processing times as MLP, while the attentional network, GATv2, is somewhat slower. The shallow embedding model, DeepWalk, is by far the slowest. Runtimes for the neural models can often be substantially reduced with GPU acceleration.

The theory in Section 4.1 indicates that more flexible models should do better on low homophily graphs and the more rigid ones should outperform on high homophily graphs. This is illustrated in Table 4, which provides node classification accuracy scores for each model under each training condition.

The models GCN and GATv2 both use sum-based neighbor aggregation (summing over the neighbors and the node itself) as in Equation (25). The difference is that GCN uses fixed, degree-normalized weights, whereas GATv2 uses feature-adaptive attention weights. In our experiments on low homophily graphs, GATv2 outperforms GCN, consistent with attention down-weighting uninformative neighbors. GraphSAGE aggregates neighbors but concatenates the neighbor summary with the node’s own features before applying the transformation ϕ (see Equation (26) and Table 1). Concatenation creates a separate self channel in ϕ that reduces reliance on potentially conflicting neighbor messages on low homophily graphs. Accordingly, we observe GraphSAGE outperforming GATv2 in that regime. On high homophily graphs, where neighbor information aligns with labels and smoothing is beneficial, GCN (and GATv2) surpass GraphSAGE, with GCN performing best in our runs.

Finally, the advantage of GNNs over an MLP is most pronounced on high homophily datasets with scarce labels, because they can use edges to

Table 4: Average node classification accuracy of off-the-shelf GNN architectures on low and high homophily dataset collections with 1% and 80% of node labels for training.

Average Node Classification Accuracy of Default Designs				
Model Name	80% Training		1% Training	
	High Homophily	Low Homophily	High Homophily	Low Homophily
GCN	85.31	37.69	72.57	30.99
GATv2	86.95	44.89	69.41	30.93
GraphSAGE	83.77	56.69	65.22	34.33
MLP	81.67	54.97	46.52	33.89
DeepWalk	81.11	33.97	69.67	24.70

share information that MLP ignores. Deepwalk, which captures structural proximity only, performs comparably to GNNs on high homophily graphs but degrades on low homophily ones.

At first sight, it seems odd that MLP, which does not use edge information, tends to do better on high homophily graphs than low homophily ones. However, our low homophily datasets are also harder in several other ways. Cornell and Wisconsin are much smaller, so the MLP sees far fewer labeled examples. Actor and Squirrel have low SNR, making their node attributes difficult to learn from. Finally, Chameleon, Crocodile, and Squirrel have relatively large feature dimensions, which further complicates learning in the low-label regime. Specifically, define the relative attribute dimension for a dataset compared to the high homophily datasets by

$$r_{\text{feat}}(D) = \frac{\text{Attribute Dimension for } D}{\text{Median attribute size of high homophily group}}. \quad (57)$$

For the Chameleon, Crocodile, and Squirrel datasets, the values of $r_{\text{feat}}(D)$ are 2.2, 9.2, 2.2, respectively.

5.2 GNN width for node classification

In high homophily settings, increasing the hidden dimension d (i.e. width) often helps up to a point: additional channels can represent more label-aligned patterns until gains by this mechanism saturate at a large d . By contrast, on low homophily graphs increasing d does not address the core issue that neighbor aggregation mixes incompatible signals. Here, using a smaller d makes it less likely that the network will overfit to noise in the input, whereas a larger d only helps when combined with a mechanism that

Table 5: The average improvement of node classification accuracy for the tuned designs over the default ones on high and low dataset collections, with 1% and 80% of node labels for training.

Average Node Classification Accuracy Improvement over Default Designs				
Model Name	80% Training High Homophily	80% Training Low Homophily	1% Training High Homophily	1% Training Low Homophily
Difficulty	Easy	Medium	Medium	Hard
GCN-64	+2.59	+1.47	+3.57	+0.97
GCN-128	+2.86	+1.65	+3.89	+0.28
GCN-256	+2.67	+1.20	+4.41	+0.78
GATv2-64	+0.18	+0.26	+2.72	−0.44
GATv2-128	+0.28	+0.66	+2.73	+0.96
GATv2-256	+0.11	+0.33	+2.16	−0.10
GraphSAGE-64	+4.12	+1.22	+6.27	+1.44
GraphSAGE-128	+4.59	+0.58	+6.67	+1.32
GraphSAGE-256	+4.53	+0.62	+7.53	+0.13

temper mixing, like attention. Empirically, we observe little change in node-classification accuracy as d increases.

We divide the dataset/training configurations into three conditions: the *easy* condition is where the graph is high homophily with 80% training data, the *hard* condition is where the graph is low homophily with only 1% training data, and the other two configurations are the *medium* difficulty condition. The most benefit from tuning the hidden dimensions occurs in medium difficulty conditions.

5.3 GNN depth for node classification

Unlike in the width experiments, all encoders in this section use batch normalization after each intermediate layer. We focus on metrics that quantify oversmoothing and oversquashing; batch normalization improves the numerical stability of these metrics and makes trends more comparable across datasets.

Figure 3 shows test accuracy as a function of the number of message-passing layers: on high homophily graphs, accuracy initially increases with depth and then declines, whereas on low homophily graphs it decreases monotonically. This pattern is consistent with prior observations that, without additional architectural modifications, the node-classification accuracy of vanilla GNNs such as GCN, GraphSAGE, and GATv2 degrades once their depth exceeds only a few message-passing layers [66, 122]. In the next two

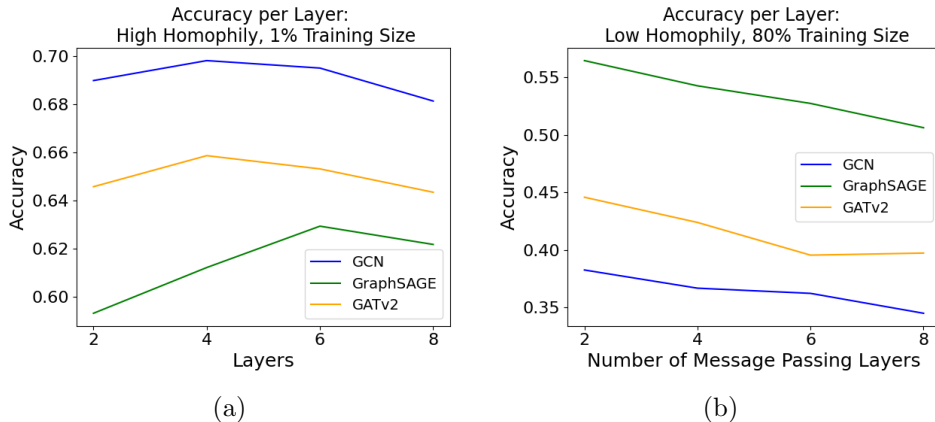


Figure 3: Test accuracy as a function of GNN depth for three architectures (GCN, GraphSAGE, GATv2). Left: high homophily graphs with 1% of nodes labeled for training. Accuracy is relatively stable across depth, with a mild peak around 4–6 layers and decreasing to 8 layers. Right: low homophily graphs with 80% labeled nodes. Here accuracy degrades steadily for all models as depth increases.

sections we investigate oversmoothing and oversquashing as complementary explanations for this decline in performance with depth.

5.3.1 Oversmoothing

The encoder in a node-classification pipeline should produce embeddings whose class means are well separated, while avoiding both excessive within-class variance and low-dimensional collapse (oversmoothing). Both phenomena shrink decision margins, but for different reasons. In over-dispersion, excessive within-class variance increases overlap between classes, reducing margins. In oversmoothing, repeated neighborhood aggregation pushes embeddings into a shared low-dimensional subspace: class means move closer together and within-class variation is confined to only a few directions. As a result, many nodes lie near decision boundaries and the remaining directions become overly influential; small noise or bias along these directions is amplified, making nodes harder to distinguish when these directions misalign with class structure. This intuition is borne out empirically: the 5th-percentile nearest-centroid margins monotonically shrink with depth beyond four layers, indicating that an increasing fraction of nodes lie close to decision boundaries (see Supplementary Material, Section S.1.3)

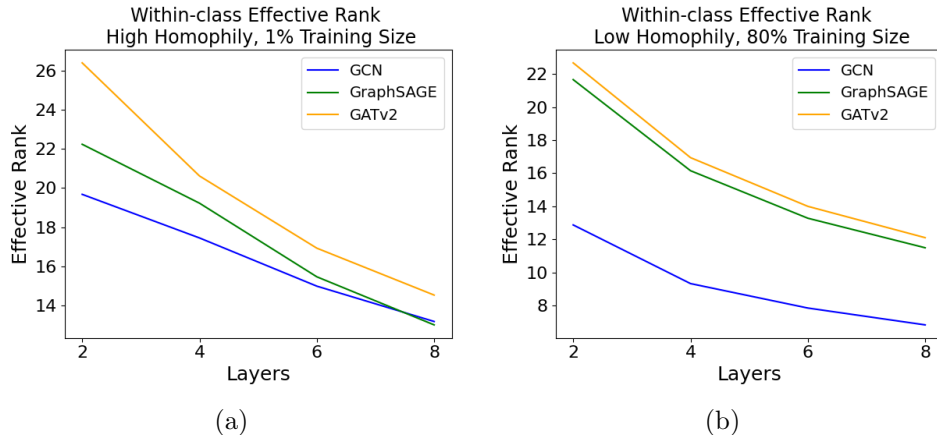


Figure 4: Within-class effective rank of node embeddings as a function of GNN depth for three architectures. The within-class effective rank is an entropy-based measure of the effective subspace dimension of the covariance of within-class residuals; lower values indicate stronger representation collapse (oversmoothing). Left: high homophily graphs with 1% of nodes labeled for training. Right: low homophily graphs with 80% of nodes labeled for training. In both regimes, effective rank decreases steadily with depth for all models, with GATv2 retaining the most within-class dimensionality, GraphSAGE in between, and GCN collapsing fastest.

Classical oversmoothing proxies, such as Dirichlet energy of the node features, track the similarity of neighboring node features, but in our experiments (and in line with [34]) their trends were relatively weak compared to dimension-based diagnostics. We therefore quantify collapse using the *Within-Class Effective Rank*, which measures how many independent directions of variation remain once class means have been removed; lower values indicate stronger oversmoothing. See Supplementary Material, Section S.1.2 for a formal definition.

This metric is an instance of the standard entropy-defined effective rank of a matrix [94], which is widely used as an entropy-based measure of effective dimensionality in statistics and deep learning [35, 41, 148]. Recent work has proposed using the effective or numerical rank of GNN feature representations as an oversmoothing metric [34], but, to our knowledge, applying effective rank specifically to the covariance of within-class residuals as an oversmoothing diagnostic is novel.

Comparison of three architectures. In Figure 4, the effective-rank values for all three models, GCN, GATv2, and GraphSAGE, decrease monotonically as the number of layers increases, falling between 6 and 11 units depending on the model. This is expected because aggregation in message passing layers are smoothing operations (repeated averaging), which reduces node feature diversity, both globally and within classes. The effective rank scores fall accordingly.

The relative scores align with architectural differences. The GCN architecture uses a degree-normalized formula for averaging in its aggregation, which averages rather uniformly. Diversity is depressed already at layer two, and it falls at a steady rate. The GraphSAGE architecture is different than GCN in that it concatenates a node’s own feature vector with an aggregation of its neighbors instead of summing them, see Table 1, page 15. This helps it preserve the diversity of the graph’s original node feature signal for longer than using pure summation. Its effective rank scores therefore trend higher than the rank scores of GCN. The GATv2 architecture uses an attention mechanism that can sustain within-class node feature diversity when node features are varied, resulting in high values in shallower networks. Its aggregation is also a smoothing operation, causing node feature vectors to look increasingly similar as the layers increase. As the inputs to the attention mechanism become more similar, the attention weights also become more similar. Aggregation of neighborhood node features then becomes more uniform, behaving closer to that of GCN, leading to a larger drop with depth.

These mechanisms do not change with the homophily of the graph, so it is expected that we see the same basic shape and model ordering in both the high and low homophily graphs. The main difference is scale. We observe the largest drop with GCN because it is the least able to avoid mixing incompatible neighborhood node features. GraphSAGE and GATv2 scores drop the least because of the self channel and the ability to weight neighbors, respectively, helps them better avoid mixing incompatible feature vectors.

Skip connections. Figure 5 provides plots of GCN under three skip strategies: no skips, residual skip-sum, and Jumping Knowledge (JK). Residual skip-sum adds each layer’s output to its input, preserving an identity pathway allowing each layer to learn only a refinement of the previous layer’s node feature vector instead of an entirely new one [49, 17]. We use the maximizing variant of Jumping Knowledge, JK-max, which takes the element-wise maximum feature vector across all layers of the network [122].

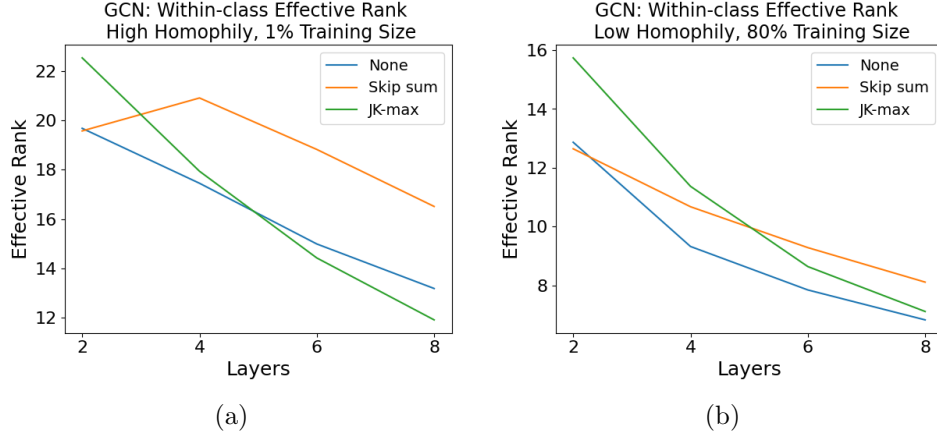


Figure 5: Within-class effective rank of GCN node embeddings as a function of depth for different skip-connection schemes. The within-class effective rank is an entropy-based measure of the effective subspace dimension of the covariance of within-class residuals; lower values indicate stronger representation collapse. Left: high homophily graphs with 1% of nodes labeled for training. Right: low homophily graphs with 80% labeled. In both regimes, the no-skip model collapses fastest, skip-sum consistently preserves the most within-class dimensionality at larger depths, and JK-max starts with the highest rank at shallow depth but quickly converges toward the no-skip baseline as depth increases.

Residual skip-sum best preserves within-class directions in our experiments. Effective rank values grow over the first layers of the network, peaking at four layers, and then decline as repeated aggregations creates redundancy. Jumping Knowledge with max pooling preserves sharp differences in node feature vectors because max pooling does not average them away. However, as with any architecture that uses message passing, aggregation is a smoothing operation: repeated averaging with neighbors makes node features less sharp and more similar, pulling feature values toward local averages. As depth increases, the early layers therefore tend to have sharper features with larger maxima, and Jumping Knowledge keeps selecting those early-layer features, so the final representation relies on a limited set of directions coming predominantly from the early layers. Those early-layer representations also continue to smooth during training. Empirically, the within-class effective-rank values for Jumping Knowledge eventually fall more quickly than those for the residual skip-sum. The no-skip method

exhibits the strongest smoothing overall.

5.3.2 Oversquashing

Given a node n , we define a Jacobian-based oversquashing diagnostic, Message Squeeze per Bridge Edge (MSBE), as follows. Let

$$B_r(n) = \{u : \text{dist}(u, n) \leq r\} \quad (58)$$

be the radius r ball centered at n , where dist is the shortest path distance. Let $\mathcal{E}$ be the set of graph edges and define

$$\partial B_r = \{(u, v) \in \mathcal{E} : u \in B_r(n), v \notin B_r(n) \text{ or vice versa}\}, \quad (59)$$

be the set of bridge edges crossing the boundary of the ball. Then

$$\text{MSBE}_r(n) = \frac{1}{|\partial B_r(n)| + \varepsilon} \sum_{u \notin B_r(n)} J(u \rightarrow n), \quad (60)$$

where $J(u \rightarrow n)$ is the L^2 Jacobian influence from a far-away node u to n . See Section S.1.4 for details. Intuitively MSBE quantifies, on average, how much far-node signal must traverse each bridge edge in $\partial B_r(n)$ in order to affect n . Because different nodes and datasets can have very different numbers of boundary-crossing bridge edges, dividing by $|\partial B_r(n)|$ normalizes for local expansion: in regions with many bridge edges, the same far-node signal can enter $B_r(n)$ through more routes, while in bottlenecked regions with few bridge edges it is concentrated across a smaller cut.

To our knowledge, prior formalizations of oversquashing either use node-to-node sensitivity aggregated over nodes without explicit normalization by boundary size [105, 37, 15] or are model agnostic, purely structural bottleneck measurements such as Cheeger-type expansion, curvature, or effective resistance [10, 105, 15]. MSBE is, to our knowledge, the first model-aware diagnostic that combines node-to-node influence with an expander-style normalization, yielding locally defined measurements that are comparable across datasets.

Experimental results. In our experiments, we set $r = 2$, and average MSBE over 256 sampled nodes. Notice that for GNNs that have $L > r$ layers, MSBE usually increases with L : each additional layer expands the receptive field of n to include more nodes outside $B_r(n)$, while the number of bridge edges $|\partial B_r(n)|$ for the chosen r is fixed.

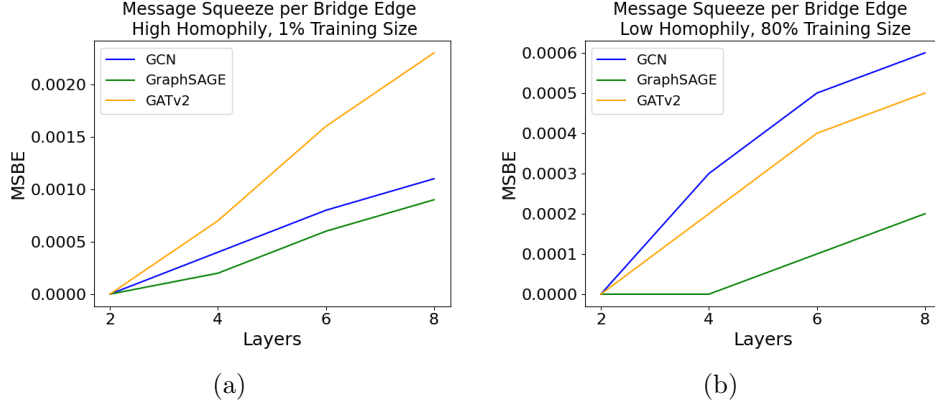


Figure 6: Message Squeeze per Bridge Edge (MSBE) as a function of GNN depth for GCN, GraphSAGE, and GATv2, averaged over the high homophily graphs with 1% training labels (left) and the low homophily graphs with 80% training labels (right). In both regimes, MSBE increases with depth (indicating stronger oversquashing), with GATv2 (respectively GCN) attaining the highest MSBE on high- (respectively low-) homophily graphs and GraphSAGE consistently lowest. MSBE values are noticeably larger in the high homophily setting.

The model ordering in Figure 6 aligns with the aggregation mechanisms. The sum-based update of GCN and GATv2 causes more neighbor information to be imported at each layer, greater cross-boundary flow, and a higher MSBE score compared to GraphSAGE, which uses the concatenation-based update (See $\wedge$ in Table 1 on page 15.)

The qualitative depth trend and ordering are the same in both the high and low homophily regimes; the difference is primarily in scale. In our benchmark, high homophily graphs tend to have fewer cross-neighborhood edges than low homophily graphs, so, all else being equal, the per-edge squeeze (MSBE) is higher. On low homophily graphs, the learned architectures often down-weight conflicting neighbor features and rely more on self features. This is especially true for GraphSAGE, which preserves a separate self channel via concatenation, and for GATv2, whose attention can reduce neighbor weights. The GCN architecture is not flexible enough to reweight individual neighbors, but its learned transformation can shift the relative contribution of self versus neighbors on average, which also tends to reduce neighbor influence in low-homophily graphs.

Improved influence after rewiring. To assess mitigation, we use the *Change in Message Squeeze per Bridge Edge* (ΔMSBE). For a target node n , we fix its far set $F_r(n) = \mathcal{N} \setminus B_r(n)$. We then add a small number of shortcut edges from n to the M far nodes in $F_r(n)$ with the node-to-node influence $J(u \rightarrow n)$. Intuitively, these are far nodes whose signals already reach n through long, potentially distorted paths, so these are the far nodes most likely to be affected by any bottleneck. Adding shortcuts to these nodes opens those routes and reveals how much the graph structure was limiting the model’s ability to use their signal. Keeping the GNN parameters fixed, we recompute MSBE on the rewired graph and define

$$\Delta\text{MSBE}_r(n) = \frac{\sum_{u \in F_r(n)} J^+(u \rightarrow n) - \sum_{u \in F_r(n)} J(u \rightarrow n)}{|\partial B_r(n)| + \varepsilon}, \quad (61)$$

where $J^+(u \rightarrow n)$ is the influence on the rewired graph. A positive $\Delta\text{MSBE}_r(n)$ means that the same far region can now send more signal per bridge edge to n , indicating that the shortcuts relieved part of the bottleneck. See Supplementary Material, Section S.1.5 for the formal definition.

In our experiments, we compute $\Delta\text{MSBE}_r(n)$ for 128 randomly chosen nodes n , using four shortcuts ($M = 4$) and taking far nodes to be at least two hops away ($r = 2$); the results are shown in Figure 7. Two opposing effects shape $\Delta\text{MSBE}_r(n)$ as the network depth L increases. First, deeper networks can reuse the new 1-hop signals from the shortcuts across more layers, so $\Delta\text{MSBE}_r(n)$ tends to rise initially. At the same time, as we add more layers, repeated neighbor-averaging makes node representations smoother: nearby nodes look increasingly similar, and changing any single node’s features only slightly perturbs the output at a target node. In this regime, even a helpful shortcut edge produces a smaller change in far-node influence, so $\Delta\text{MSBE}_r(n)$ eventually peaks and then gradually declines with diminishing returns.

Targeted shortcuts act as persistent 1-hop inputs reused across layers. For GraphSAGE, this occurs without down-weighting existing high-weight edges (as attention does in GATv2) and with less global degree-normalization of all neighbors (as in GCN). As a result, GraphSAGE tends to show a smaller post-peak decline or a continued but gentler increase with depth—the shortcuts keep paying off, even though the slope ultimately flattens due to the same diminishing-returns effect.

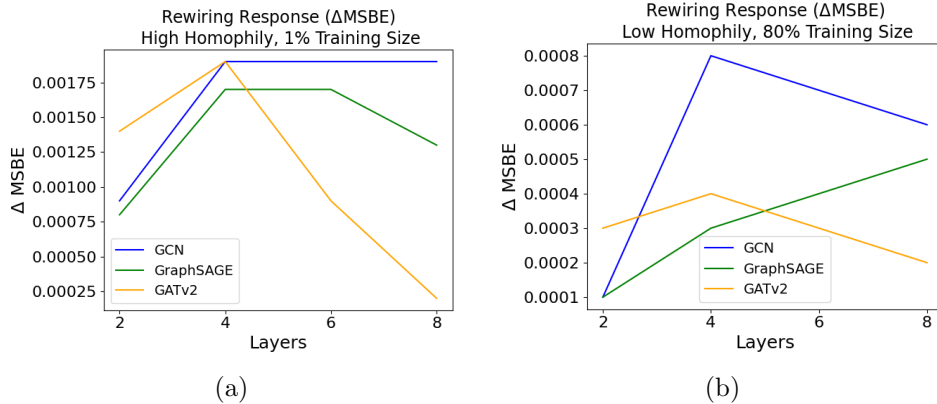


Figure 7: Rewiring response measured by the change in Message Squeeze per Bridge Edge (Δ MSBE) as a function of GNN depth for GCN, GraphSAGE, and GATv2, averaged over the high homophily graphs with 1% training labels (left) and the low homophily graphs with 80% training labels (right). For each target node we add four shortcut edges to high-influence far nodes and recompute MSBE; positive values indicate that rewiring increases far-node signal per bridge edge. In both regimes, Δ MSBE usually peaks at moderate depth and then declines, showing that the marginal benefit of local rewiring decreases as the network becomes deeper.

5.4 Training sizes for high homophily graphs

To better understand the accuracy gains that come with more training data, we compare our main oversmoothing and oversquashing metrics in the 80% and 1% training regimes on high homophily graphs. That is, we compare the within-class effective-rank and MSBE metrics. For low homophily graphs with only 1% training data, the models perform close to chance, so we omit those results.

The main observation (shown in Figure 8) is that the profiles of both metrics as a function of depth are similar for the two training sizes, but their scales differ. With more training data, the supervised loss provides enough signal to preserve multiple task-relevant directions within each class, increasing the within-class effective-rank values relative to the 1% training case; this richer class-specific structure is consistent with the higher node-classification accuracy we observe. For MSBE, with abundant labels—especially on high homophily graphs—the trained models learn to rely less on far-away nodes, which reduces long-range influence and shifts the MSBE profiles downward

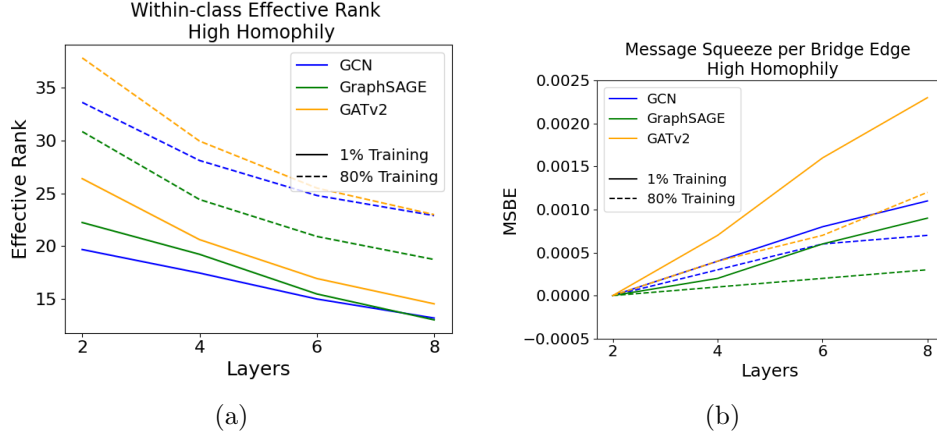


Figure 8: Within-class effective rank (left) and Message Squeeze per Bridge Edge (MSBE, right) as a function of depth for GCN, GraphSAGE, and GATv2 on high-homophily datasets. Solid lines correspond to models trained with 1% of node labels and dashed lines to models trained with 80% of node labels. With more training data, within-class effective rank remains higher, indicating reduced oversmoothing, whereas MSBE tends to be larger in the low-label regime, reflecting stronger oversquashing when supervision is scarce.

while preserving their overall shape. In combination, higher effective rank and lower MSBE indicate embeddings that are better separated and less dominated by noisy long-range messages, explaining the accuracy gains at larger training sizes.

5.4.1 Improving deeper GNNs with simple architectural tuning

In this section, we answer three following questions.

Q1. Does simple tuning actually improve accuracy? GNNs expose many design variables beyond hidden dimension. Rather than using automated hyperparameter search, we focus on a small set of simple, interpretable choices. Following You et al. [128], we fix the design variables that are shared by top-performing configurations (Table 6), and greedily tune the six remaining hyperparameters shown in Table 7: the numbers of message-passing, post-processing, and pre-processing layers, the skip-connection scheme, the aggregation function, and the learning rate. Graph-

Table 6: Hyperparameter values shared by top performing designs in You et al. [128].

Activation (PReLU)	Batch Norm (True)	Dropout (False)
Batch Size (32)	Optimizer (Adam)	Epochs (400)

Table 7: Hyperparameter values to tune

Parameter	Tuning Order	Starting Value	Options
Message-Passing Layers	1	2	1, 2, 3, 4, 5, 6, 7, 8
Post-Processing Layers	2	1	1, 2, 3
Pre-Processing Layers	3	1	1, 2, 3
Layer Connectivity	4	Skip Sum	None, Skip Sum, Skip Concatenate
Aggregation Function	5	Mean	Add, Mean, Max
Learning Rate	6	0.01	0.005, 0.01, 0.0125, 0.015

Gym [128]¹ is used to manage experiments.

For each dataset and architecture, we start from a default configuration with 128 hidden dimensions (Section 5.2), train all models for 400 epochs, and select the hyperparameter option in each row of Table 7 that maximizes the average test accuracy over 25 random splits, proceeding in the specified tuning order. As in You et al. [128], we adjust the number of hidden layers so that different designs have comparable overall size.

Table 8 reports the average accuracy improvement of the tuned designs over the default ones across four regimes (high/low edge homophily and 1%/80% training labels). Tuning provides little benefit in the easiest and hardest settings: when high homophily graphs are already easy (80% labels), and when low homophily graphs are label-scarce (1% labels), the tuned and default configurations perform similarly, and in the hardest regime the default can even be slightly better, as for GraphSAGE. A likely explanation is that the sweep starts from a generic default configuration, and in the hardest setting the signal is too weak to reliably identify hyperparameter changes that improve on that starting point. In contrast, tuning yields substantial gains in the medium-difficulty regimes, particularly on low homophily graphs with abundant labels, where GCN and GATv2 improve by roughly 23 and 16 percentage points on average, respectively.

Much of this gain from low homophily graphs comes from two small, challenging datasets, Cornell and Wisconsin, which high feature SNR. Table 9 shows that, on these datasets, tuning improves GCN and GATv2 accuracies by 22–49 percentage points, whereas improvements on the other

¹<https://github.com/snap-stanford/GraphGym.git>

Table 8: The average improvement of node classification accuracy for the tuned designs over the default design with 128 hidden dimensions (see Section 5.2) on high and low homophily dataset collections, with 1% and 80% of node labels for training. The hyperparameters for each algorithm have been tuned to each dataset.

Average Node Classification Accuracy Improvement over Default with 128 Hidden Dims				
Model Name (Difficulty)	80% Training High Homophily	80% Training Low Homophily	1% Training High Homophily	1% Training Low Homophily
	(Easy)	(Medium)	(Medium)	(Hard)
GCN	+0.57	+22.98	+0.93	−0.22
GATv2	+1.53	+16.39	+4.57	−0.92
GraphSAGE	+0.15	+6.18	+4.02	−3.89

Table 9: The average improvement of node classification accuracy for tuned designs over default ones with 128 hidden dimensions (see Section 5.2) on the collection of Cornell and Wisconsin datasets versus the collection of other low homophily datasets, with 80% of nodes training. The hyperparameters for each algorithm have been tuned to each dataset.

Node Classification Accuracy Improvement		
80% Training	Cornell & Wisconsin	Other Low Homophily
GCN	+49.32	+9.37
GATv2	+41.58	+3.80
GraphSAGE	+22.89	−2.12

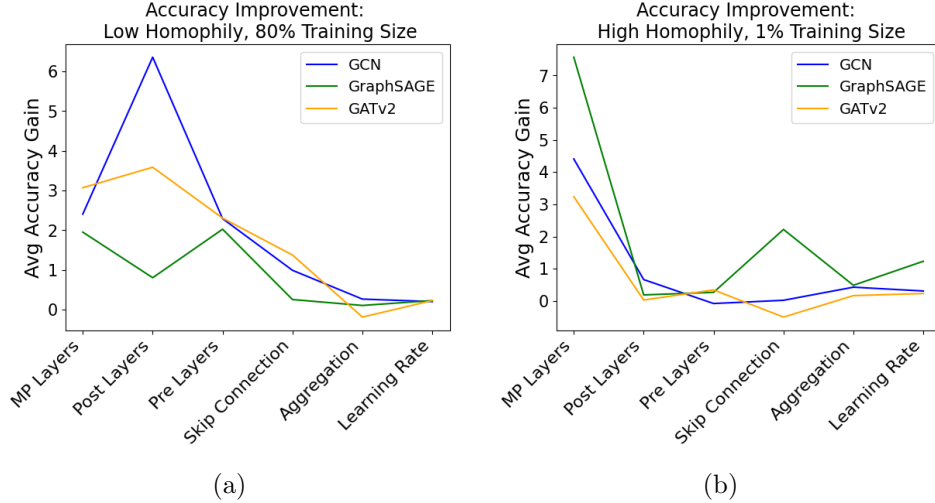


Figure 9: Following the greedy hyperparameter tuning process described in Section 5.4.1, the figures show the improved node classification accuracy after tuning the given design component of the GNN compared to the design before it was tuned.

low homophily graphs are more modest. These results indicate that simple architectural tuning—adjusting depth, pre-/post-processing layers, and skip connections, along with a few basic training hyperparameters—can significantly improve performance in regimes where the task is neither trivial nor hopeless, while offering limited benefit when the dataset is already easy or severely label-constrained.

Q2. Which architectural components matter?

Figure 9 shows the incremental accuracy gain from tuning each hyperparameter in our greedy procedure. Simple structural changes that affect how information is propagated and combined across layers (depth, pre- and post-processing layers, and skip connections) account for most of the achievable gains, whereas tuning the aggregation function and learning rate yields comparatively little improvement. Overall, the tuned configurations favor moderate depth, several pre-/post-processing layers, and non-trivial skip connections; we summarize these designs in the Supplementary Material, Section S.2.

Figure 10 illustrates the role of different layer types (pre-/post-processing and message-passing) in terms of between-class distance (“signal”) and within-

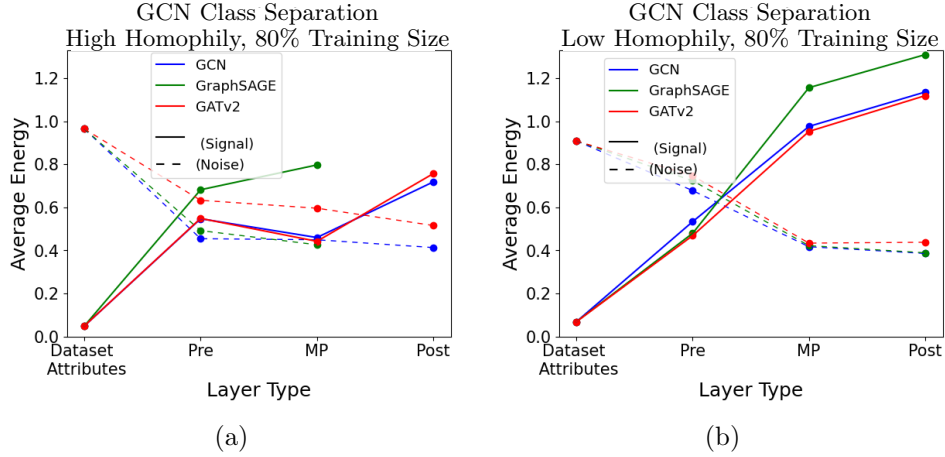


Figure 10: GCN between-class distance (“signal”) and within-class variance (“noise”) for each layer type on the low homophily datasets with 80% training size. Higher between-class distance and lower within-class variance indicate better class separation.

class variance (“noise”) of node features for the tuned architectures. The pre-processing layers transform and rescale the raw node attributes so that nodes from the same class are represented more consistently and within-class variance is reduced before message passing. This effect is visible in Figure 10: moving from the dataset attributes to the pre-processing layers lowers the noise curves in both the high and low homophily regimes, while keeping the signal at a comparable level. The message-passing layers then aggregate these features across neighbors. On high homophily graphs, this averaging mainly mixes same-class neighbors, which substantially lowers within-class variance while maintaining or even increasing between-class distance, thereby improving the signal relative to the noise. On low homophily graphs, where neighbors are more likely to have different labels, the gain from message passing is more modest, and the tuned architectures therefore favor shallower stacks of message-passing layers. Finally, the post-processing layers operate on these refined, propagated features and, in both regimes, produce the highest between-class distance without a corresponding rise in within-class variance, effectively turning the representations into well-separated decision regions.

Q3. Comparison with off-the-shelf RevGNN reference models.

We compare our tuned architectures to off-the-shelf RevGNN variants

Table 10: Node classification accuracy comparison of models on low and high homophily dataset collections, with 1% and 80% of node labels for training.

Node Classification Accuracy		
Model Name	80% Training	1% Training
	Low Homophily	High Homophily
GCN _{Tuned}	62.32	77.39
RevGCN	50.93	77.86
GATv2 _{Tuned}	61.94	76.71
RevGATv2	53.64	77.99
GraphSAGE _{Tuned}	63.45	75.91
RevSAGE	59.61	75.87

used in strong Open Graph Benchmark models (Table 10). RevGNNs are reversible GNNs built on GCN-, GAT-, or GraphSAGE-style encoders and are designed so that memory usage is essentially independent of depth [63], making them an appropriate reference for efficient deep GNN design. In our experiments, we use the RevGNN models without dataset-specific tuning, with 160 hidden dimensions, 4 layers, 200 training epochs, and an 80/10/10 train/validation/test split; accordingly, this comparison should be interpreted only as a reference against standard RevGNN configurations, not as a controlled comparison between equally tuned model families. Under that interpretation, Table 10 shows that simple tuning of vanilla GNNs can make them competitive with standard off-the-shelf RevGNN settings.

5.5 Link Prediction

We use an 80/10/10 train/validation/test split with a 50/50 balance between positive and negative edges, and we train for 200 epochs. As in the node-classification setting, all results are averages over 25 trials, and we use the same collections of high- and low-homophily datasets.

For link prediction, we remove the validation and test positive edges from the graph and run message passing only on the remaining training graph. The GNN encoder then produces a node embedding for each node using the observed graph structure and node attributes. To score a candidate edge (i, j) , we apply the decoder from Section 4.2, $\sigma(z_i^\top z_j)$, where z_i and z_j are the learned node embeddings and σ is the sigmoid function. Positive examples are observed edges, while negative examples are sampled non-edges. During validation and testing, the model is evaluated on held-out positive edges

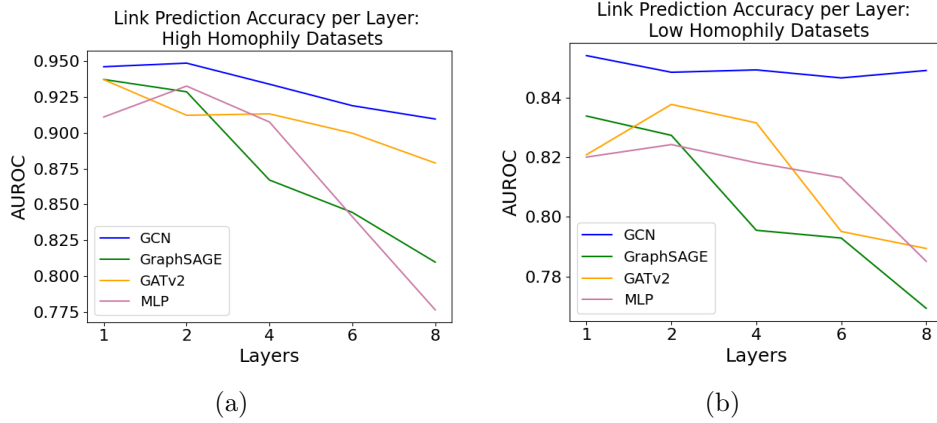


Figure 11: Link prediction performance (AUROC) as a function of the number of message-passing layers for GCN, GraphSAGE, GATv2, and an MLP baseline on high and low homophily datasets. In both regimes, AUROC typically peaks at 1–2 layers and then degrades with additional depth, consistent with oversmoothing and the propagation of noisy signals in deeper vanilla GNNs, while the MLP, which ignores edges, shows the usual decline in performance for very deep feed-forward networks.

together with sampled negative edges, so the target edges are not available to the encoder when constructing the embeddings.

With this setup, the depth trends are even more conservative than for node classification (see Figure 11). Across both high- and low-homophily graphs, performance peaks at around two message-passing layers and then flattens or declines as depth increases. Additional layers mix information from increasingly distant parts of the graph; this can help node classification by smoothing noisy labels within a class up to moderate depth, but for link prediction it tends to wash out the local structural patterns—common neighbors, short paths, and small motifs—that distinguish positive from negative node pairs. Once many nodes share very similar embeddings due to oversmoothing, the endpoints of an edge and a non-edge can become almost indistinguishable, making link prediction especially sensitive to excess depth.

Motivated by this pattern, we next fix the number of message-passing layers at two and tune only the pre- and post-processing layers. We find that the best-performing architectures in our experiments always include a single post-processing layer, whereas the benefit of a pre-processing layer is model- and regime-dependent (see Table 11). Two message-passing layers already

Table 11: Effect of pre- and post-processing layers on link prediction AUROC with 2 message-passing layers. Δ AUROC is the improvement over the baseline with no pre- or post-processing layers.

Regime	Model	Pre Layers	Post Layers	AUROC	Δ AUROC
High homo.	GCN	0	1	0.9493	+0.0095
	GraphSAGE	0	1	0.9279	+0.0034
	GATv2	1	1	0.9408	+0.0287
Low homo.	GCN	1	1	0.8525	+0.0041
	GraphSAGE	0	1	0.8318	+0.0111
	GATv2	0	1	0.8530	+0.0253

capture most of the informative local context for link prediction—immediate neighbors and their overlaps, i.e., one- and two-hop structure—so additional preprocessing is mainly helpful when the raw node attributes are noisy or poorly scaled. By contrast, the post-processing layer consistently improves performance by adding an extra nonlinearity after local information has been aggregated, sharpening the decision boundary in this already well-captured neighborhood regime.

6 Conclusion

A decade ago deep convolutional neural networks for image classification initiated a revolution where feature learning was integrated into the training process of a neural network, and this was subsequently extended to data structures like irregular graphs. The encoder-decoder framework neatly describes these models, and the shortcomings of simpler encoder-decoder models motivates the use of more complicated Graph Neural Networks (GNNs). Graph neural networks have attracted considerable attention due to state-of-the-art results on a range of graph analysis tasks and datasets, but because of the great variety of graphs and graph analysis tasks, they can be difficult to use for those new to the field. As such, we hope our overview of GNNs, their construction and behavior on a variety of datasets and training conditions, has prepared the reader to solve diverse graph problems and understand the technical aspects of literature.

Acknowledgements

This work was partially funded by The MITRE Corporation Independent Research and Development Program. After Section 5.2, OpenAI GPT 5-

class systems were used to help generate some code and draft some paragraphs, which the authors revised or rewrote. OpenAI GPT systems were also used to edit text in Sections 1, 2, 4.4, and 5 and to suggest references for Sections 2 and 5. Google Gemini (consumer web/app access) was used to help check the accuracy of some statements.

Approved for Public Release; Distribution Unlimited. Public Release Case Number 26-1038. ©2026 The MITRE Corporation. ALL RIGHTS RESERVED.

References

- [1] Khushnood Abbas, Alireza Abbasi, Shi Dong, Ling Niu, Laihang Yu, Bolun Chen, Shi-Min Cai, and Qambar Hasan. 2021. Application of network link prediction in drug discovery. *BMC Bioinformatics* 22, 1 (2021), 187. doi:10.1186/s12859-021-04082-y
- [2] Shakeel Ahmad, Muhammad Zubair Asghar, Fahad M Alotaibi, and Irfanullah Awan. 2019. Detection and classification of social media-based extremist affiliations using sentiment analysis techniques. *Human-centric Computing and Information Sciences* 9, 1 (2019), 1–23.
- [3] Amr Ahmed, Nino Shervashidze, Shravan Narayanamurthy, Vanja Josifovski, and Alexander J Smola. 2013. Distributed large-scale natural graph factorization. In *Proceedings of the 22nd international conference on World Wide Web*. 37–48.
- [4] Luca Maria Aiello, Alain Barrat, Rossano Schifanella, Ciro Cattuto, Benjamin Markines, and Filippo Menczer. 2012. Friendship prediction and homophily in social media. *ACM Transactions on the Web (TWEB)* 6, 2 (2012), 1–33.
- [5] S. Akansha. 2025. Over-squashing in graph neural networks: a comprehensive survey. *Neurocomputing* 642 (2025), 130389. <https://www.sciencedirect.com/science/article/pii/S0925231225010616>
- [6] Uri Alon and Eran Yahav. 2021. On the Bottleneck of Graph Neural Networks and its Practical Implications. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=i800Ph0CVH2>

- [7] Siddhant Arora. 2020. A survey on graph neural networks for knowledge graph completion. *arXiv preprint arXiv:2007.12374* (2020).
- [8] Muhammet Balcilar, Guillaume Renton, Pierre Hérault, Benoit Gauzere, Sébastien Adam, and Paul Honeine. 2020. Bridging the gap between spectral and spatial domains in graph neural networks. *arXiv preprint arXiv:2003.11702* (2020).
- [9] Sambaran Bandyopadhyay and Vishal Peter. 2021. Unsupervised constrained community detection via self-expressive graph neural network. In *Uncertainty in Artificial Intelligence*. PMLR, 1078–1088.
- [10] Pradeep Kr Banerjee, Kedar Karhadkar, Yu Guang Wang, Uri Alon, and Guido Montúfar. 2022. Oversquashing in GNNs through the lens of information contraction and graph expansion. In *2022 58th Annual Allerton Conference on Communication, Control, and Computing (Allerton)*. IEEE, 1–8.
- [11] Peter Battaglia, Razvan Pascanu, Matthew Lai, Danilo Jimenez Rezende, and Koray kavukcuoglu. 2016. Interaction Networks for Learning about Objects, Relations and Physics. In *Proceedings of the 30th International Conference on Neural Information Processing Systems (Barcelona, Spain) (NIPS’16)*. Curran Associates Inc., Red Hook, NY, USA, 4509–4517.
- [12] Peter W Battaglia, Jessica B Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, et al. 2018. Relational inductive biases, deep learning, and graph networks. *arXiv preprint arXiv:1806.01261* (2018).
- [13] Mikhail Belkin and Partha Niyogi. 2003. Laplacian eigenmaps for dimensionality reduction and data representation. *Neural computation* 15, 6 (2003), 1373–1396.
- [14] Smriti Bhagat, Graham Cormode, and S Muthukrishnan. 2011. Node classification in social networks. In *Social network data analytics*. Springer, 115–148.
- [15] Mitchell Black, Zhengchao Wan, Amir Nayyeri, and Yusu Wang. 2023. Understanding oversquashing in GNNs through the lens of effective resistance. In *International Conference on Machine Learning*. PMLR, 2528–2547.

- [16] Aleksandar Bojchevski and Stephan Günnemann. 2018. Deep Gaussian Embedding of Graphs: Unsupervised Inductive Learning via Ranking. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=r1ZdKJ-OW>
- [17] Xavier Bresson and Thomas Laurent. 2017. Residual Gated Graph ConvNets. *arXiv preprint arXiv:1711.07553* (2017).
- [18] Shaked Brody, Uri Alon, and Eran Yahav. 2022. How Attentive are Graph Attention Networks?. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=F72ximsx7C1>
- [19] Michael M Bronstein, Joan Bruna, Taco Cohen, and Petar Veličković. 2021. Geometric deep learning: Grids, groups, graphs, geodesics, and gauges. *arXiv preprint arXiv:2104.13478* (2021).
- [20] Joan Bruna, Wojciech Zaremba, Arthur Szlam, and Yann LeCun. 2013. Spectral networks and locally connected networks on graphs. *arXiv preprint arXiv:1312.6203* (2013).
- [21] Margarita Bugue no, Russa Biswas, and Gerald de Melo. 2024. Graph-Based Explainable AI: A Comprehensive Survey. *HAL Open Science* (2024). <https://hal.science/hal-04660442/>
- [22] Michael Burch, Marcel Hlawatsch, and Daniel Weiskopf. 2017. Visualizing a sequence of a thousand graphs (or even more). In *Computer Graphics Forum*, Vol. 36. Wiley Online Library, 261–271.
- [23] Joshua G. Burkhart, Guanming Wu, Xubo Song, Francesco Raimondi, Shannon McWeeney, Melissa H. Wong, and Youping Deng. 2023. Biology-inspired Graph Neural Network Encodes Reactome and Reveals Biochemical Reactions of Disease. *Patterns* 4, 7 (2023), 100758. doi:10.1016/j.patter.2023.100758
- [24] Lei Cai, Jundong Li, Jie Wang, and Shuiwang Ji. 2021. Line graph neural networks for link prediction. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2021).
- [25] Shaosheng Cao, Wei Lu, and Qionghai Xu. 2015. Grarep: Learning graph representations with global structural information. In *Proceedings of the 24th ACM international on conference on information and knowledge management*. 891–900.

- [26] Mateusz Cedro and David Martens. 2025. GraphXAIN: Narratives to Explain Graph Neural Networks. In *Proceedings of the World Conference on Explainable Artificial Intelligence (XAI)*, Vol. 2578. Springer Nature, 91–114. https://link.springer.com/chapter/10.1007/978-3-032-08327-2_5
- [27] Ines Chami, Sami Abu-El-Haija, Bryan Perozzi, Christopher Ré, and Kevin Murphy. 2022. Machine learning on graphs: A model and comprehensive taxonomy. *Journal of Machine Learning Research* 23, 89 (2022), 1–64.
- [28] Guangyi Chen and Zhi-Ping Liu. 2022. Graph attention network for link prediction of gene regulations from single-cell RNA-sequencing data. *Bioinformatics* 38, 19 (2022), 4522–4529. doi:10.1093/bioinformatics/btac559
- [29] Ming Chen, Zhewei Wei, Zengfeng Huang, Bolin Ding, and Yaliang Li. 2020. Simple and Deep Graph Convolutional Networks. In *Proceedings of the 37th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 119)*. 1725–1735.
- [30] Zhengdao Chen, Lisha Li, and Joan Bruna. 2020. Supervised Community Detection with Line Graph Neural Networks. In *International conference on learning representations*.
- [31] Petr Chunaev. 2020. Community detection in node-attributed social networks: a survey. *Computer Science Review* 37 (2020), 100286.
- [32] Enyan Dai, Shijie Zhou, Zhimeng Guo, and Suhan Wang. 2022. Label-Wise Graph Convolutional Network for Heterophilic Graphs. In *Proceedings of the First Learning on Graphs Conference (Proceedings of Machine Learning Research, Vol. 198)*. 26:1–26:21.
- [33] Michaël Defferrard, Xavier Bresson, and Pierre Vandergheynst. 2016. Convolutional neural networks on graphs with fast localized spectral filtering. *Advances in neural information processing systems* 29 (2016), 3844–3852.
- [34] Piero Deidda, Kaicheng Zhang, Desmond J. Higham, and Francesco Tudisco. 2025. Rethinking Oversmoothing in Graph Neural Networks: A Rank-Based Perspective. *arXiv preprint arXiv:2502.04591* (2025).

- [35] Marco Del Giudice. 2021. Effective Dimensionality: A Tutorial. *Multivariate Behavioral Research* 56, 3 (2021), 527–542. doi:10.1080/00273171.2020.1743631
- [36] Austin Derrow-Pinion, Jennifer She, David Wong, Oliver Lange, Todd Hester, Luis Perez, Marc Nunkesser, Seongjae Lee, Xueying Guo, Brett Wiltshire, et al. 2021. Eta prediction with graph neural networks in google maps. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*. 3767–3776.
- [37] Francesco Di Giovanni, Lorenzo Giusti, Federico Barbero, Giulia Luise, Pietro Liò, and Michael M. Bronstein. 2023. On Over-squashing in message passing neural networks: the impact of width, depth, and topology. In *Proceedings of the international conference on machine learning*, Vol. 202. 7865–7885. <https://proceedings.mlr.press/v202/di-giovanni23a.html>
- [38] Santo Fortunato and Darko Hric. 2016. Community detection in networks: A user guide. *Physics reports* 659 (2016), 1–44.
- [39] Victor Fung, Jiaxin Zhang, Eric Juarez, and Bobby G Sumpter. 2021. Benchmarking graph neural networks for materials chemistry. *npj Computational Materials* 7, 1 (2021), 84.
- [40] Chen Gao, Yu Zheng, Nian Li, Yinfeng Li, Yingrong Qin, Jinghua Piao, Yuhan Quan, Jianxin Chang, Depeng Jin, Xiangnan He, et al. 2023. A survey of graph neural networks for recommender systems: Challenges, methods, and directions. *ACM Transactions on Recommender Systems* 1, 1 (2023), 1–51.
- [41] Quentin Garrido, Randall Balestriero, Laurent Najman, and Yann LeCun. 2023. RankMe: Assessing the Downstream Performance of Pre-trained Self-Supervised Representations by Their Rank. In *Proceedings of the 40th International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 202)*. 10929–10974. arXiv:2210.02885.
- [42] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. 2017. Neural Message Passing for Quantum Chemistry. In *Proceedings of the 34th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 70)*, Doina Precup and Yee Whye Teh (Eds.). PMLR, 1263–1272. <https://proceedings.mlr.press/v70/gilmer17a.html>

- [43] Flavio Giorgi, Cesare Campagnano, Fabrizio Silvestri, and Gabriele Tolomei. 2025. Natural Language Counterfactual Explanations for Graphs Using Large Language Models. *arXiv preprint arXiv:2410.09295v2* (2025).
- [44] Cheng-Hua Gong, Yao Cheng, Jian-Xiang Yu, Can Xu, Cai-Hua Shan, Si-Qiang Luo, and Xiang Li. 2026. A survey on learning from graphs with heterophily: Recent advances and future directions. *Frontiers of Computer Science* 20, 2 (2026), 2002314.
- [45] Aditya Grover and Jure Leskovec. 2016. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*. 855–864.
- [46] William L Hamilton. 2020. Graph representation learning. *Synthesis Lectures on Artificial Intelligence and Machine Learning* 14, 3 (2020), 1–159.
- [47] William L Hamilton, Rex Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*. 1025–1035.
- [48] William L Hamilton, Rex Ying, and Jure Leskovec. 2017. Representation learning on graphs: Methods and applications. *arXiv preprint arXiv:1709.05584* (2017).
- [49] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 770–778. doi:10.1109/CVPR.2016.90
- [50] Yinhan He, Zaiyi Zheng, Patrick Soga, Yaochen Zhu, Yushun Dong, and Jundong Li. 2024. Explaining Graph Neural Networks with Large Language Models: A Counterfactual Perspective for Molecular Property Prediction. *arXiv preprint arXiv:2410.15165v1* (2024).
- [51] Di Jin, Ziyang Liu, Weihao Li, Dongxiao He, and Weixiong Zhang. 2019. Graph convolutional networks meet markov random fields: Semi-supervised community detection in attribute networks. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 33. 152–159.

- [52] Yufei Jin and Xingquan Zhu. 2025. Oversmoothing alleviation in graph neural networks: a survey and unified view. *Knowledge and Information Systems* (2025), 1–27.
- [53] Shixiong Jing, Lingwei Chen, Quan Li, and Dinghao Wu. 2024. H² GNN: Graph Neural Networks with Homophilic and Heterophilic Feature Aggregations. In *International Conference on Database Systems for Advanced Applications*. Springer, 342–352.
- [54] Kazi Zainab Khanam, Gautam Srivastava, and Vijay Mago. 2023. The homophily principle in social network analysis: A survey. *Multimedia Tools and Applications* 82, 6 (2023), 8811–8854.
- [55] Dongkwan Kim and Alice Oh. 2021. How to Find Your Friendly Neighborhood: Graph Attention Design with Self-Supervision. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=Wi5KUNlqWty>
- [56] Jongmin Kim, Taesup Kim, Sungwoong Kim, and Chang D Yoo. 2019. Edge-labeling graph neural network for few-shot learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 11–20.
- [57] Thomas Kipf, Ethan Fetaya, Kuan-Chieh Wang, Max Welling, and Richard Zemel. 2018. Neural relational inference for interacting systems. In *International Conference on Machine Learning*. PMLR, 2688–2697.
- [58] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=SJU4ayYgl>
- [59] Konstantin Klemmer, Nathan S Safir, and Daniel B Neill. 2023. Positional encoder graph neural networks for geographic data. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 1379–1389.
- [60] Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix factorization techniques for recommender systems. *Computer* 42, 8 (2009), 30–37.

- [61] Mert Kosan, Zexi Huang, Sourav Medya, Sayan Ranu, and Ambuj Singh. 2025. GCFExplainer: Global Counterfactual Explainer for Graph Neural Networks. *ACM Transactions on Intelligent Systems and Technology* 16, 5 (2025). <https://dl.acm.org/doi/pdf/10.1145/3698108>
- [62] Bin Li and Sheida Nabavi. 2024. A Multimodal Graph Neural Network Framework for Cancer Molecular Subtype Classification. *BMC Bioinformatics* 25, 27 (2024), 1–15. doi:10.1186/s12859-023-05622-4
- [63] Guohao Li, Matthias Müller, Bernard Ghanem, and Vladlen Koltun. 2021. Training graph neural networks with 1000 layers. In *International conference on machine learning*. PMLR, 6437–6449.
- [64] Guohao Li, Matthias Müller, Ali Thabet, and Bernard Ghanem. 2019. DeepGCNs: Can GCNs go as deep as CNNs?. In *Proceedings of the IEEE/CVF international conference on computer vision*. 9267–9276.
- [65] Pan Li, Yanbang Wang, Hongwei Wang, and Jure Leskovec. 2020. Distance encoding: Design provably more powerful neural networks for graph representation learning. *Advances in Neural Information Processing Systems* 33 (2020), 4465–4478.
- [66] Qimai Li, Zhichao Han, and Xiao-Ming Wu. 2018. Deeper Insights into Graph Convolutional Networks for Semi-Supervised Learning. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*. AAAI Press, 3538–3545. doi:10.1609/aaai.v32i1.11604
- [67] Xuyan Li, Jie Wang, and Zheng Yan. 2025. Can Graph Neural Networks be Adequately Explained? A Survey. *Comput. Surveys* 57, 5 (2025). <https://doi.org/10.1145/3711122>
- [68] Yingqi Li, Xiaochuan Sun, Haijun Zhang, Zhigang Li, Linlin Qin, Chenwei Sun, and Zhanlin Ji. 2021. Cellular Traffic Prediction via a Deep Multi-Reservoir Regression Learning Network for Multi-Access Edge Computing. *IEEE Wireless Communications* 28, 5 (2021), 13–19.
- [69] Yaguang Li, Rose Yu, Cyrus Shahabi, and Yan Liu. 2018. Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting. In *International Conference on Learning Representations*.

- [70] Xin Liu, Jiale Yu, Siyu Tao, Beiyuan Yang, Shike Wang, Lin Wang, Fang Bai, and Jie Zheng. 2022. PiLSL: pairwise interaction learning-based graph neural network for synthetic lethality prediction in human cancers. *Bioinformatics* 38, Supplement_1 (2022), ii106–ii112. doi:10.1093/bioinformatics/btac476
- [71] Zhen Liu, Ning Zhang, Ming Yang, Jian Ren, Ran Li, Mingjun Guo, and Shusen Li. 2023. Identifying Cancer Driver Genes Based on Multi-view Heterogeneous Graph Convolutional Network and Self-attention Mechanism. *BMC Bioinformatics* 24, 16 (2023), 1–18. doi:10.1186/s12859-023-05140-3
- [72] Ana Lucic, Maartje ter Hoeve, Gabriele Tolomei, Maarten de Rijke, and Fabrizio Silvestri. 2022. CF-GNNExplainer: Counterfactual Explanations for Graph Neural Networks. In *Proceedings of the 25th International Conference on Artificial Intelligence and Statistics (AISTATS)*, Vol. 151. MLResearch Press, 4499–4511. <https://proceedings.mlr.press/v151/lucic22a.html>
- [73] Liheng Ma, Reihaneh Rabbany, and Adriana Romero-Soriano. 2021. Graph attention networks with positional embeddings. In *Pacific-Asia conference on knowledge discovery and data mining*. Springer, 514–527.
- [74] Yi Ma, Jianye Hao, Yaodong Yang, Han Li, Junqi Jin, and Guangyong Chen. 2019. Spectral-based graph convolutional network for directed graphs. *arXiv preprint arXiv:1907.08990* (2019).
- [75] Santhosh Maddila, Somula Ramasubbareddy, and K Govinda. 2020. Crime and Fraud detection using clustering techniques. *Innovations in Computer Science and Engineering* (2020), 135–143.
- [76] Russell Merris. 1994. Laplacian matrices of graphs: a survey. *Linear algebra and its applications* 197 (1994), 143–176.
- [77] Federico Monti, Davide Boscaini, Jonathan Masci, Emanuele Rodolà, Jan Svoboda, and Michael M Bronstein. 2017. Geometric Deep Learning on Graphs and Manifolds Using Mixture Model CNNs. In *CVPR*.
- [78] Mark EJ Newman. 2006. Modularity and community structure in networks. *Proceedings of the National Academy of Sciences* 103, 23 (2006), 8577–8582.

- [79] Andrew Y Ng, Michael I Jordan, and Yair Weiss. 2002. On spectral clustering: Analysis and an algorithm. In *Advances in Neural Information Processing Systems*. 849–856.
- [80] Khang Nguyen, Hieu Nong, Vinh Nguyen, Nhat Ho, Stanley Osher, and Tan Nguyen. 2023. Revisiting over-smoothing and over-squashing using ollivier-ricci curvature. In *Proceedings of the 40th International Conference on Machine Learning (Honolulu, Hawaii, USA) (ICML’23)*. Article 1080, 24 pages.
- [81] Maximilian Nickel, Kevin Murphy, Volker Tresp, and Evgeniy Gabrilovich. 2015. A review of relational machine learning for knowledge graphs. *Proc. IEEE* 104, 1 (2015), 11–33.
- [82] Mingdong Ou, Peng Cui, Jian Pei, Ziwei Zhang, and Wenwu Zhu. 2016. Asymmetric transitivity preserving graph embedding. In *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*. 1105–1114.
- [83] Bo Pan, Zhen Xiong, Guanchen Wu, Zheng Zhang, Yuntong Hu, Yifei Zhang, and Liang Zhao. 2025. GraphNarrator: Generating Textual Explanations for Graph Neural Networks. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, Vol. 1. Association for Computational Linguistics, 23–42. <https://aclanthology.org/2025.acl-long.2>
- [84] Shirui Pan, Ruiqi Hu, Guodong Long, Jing Jiang, Lina Yao, and Chengqi Zhang. 2018. Adversarially regularized graph autoencoder for graph embedding. In *International Joint Conference on Artificial Intelligence 2018*. Association for the Advancement of Artificial Intelligence (AAAI), 2609–2615.
- [85] Babita Pandey, Praveen Kumar Bhanodia, Aditya Khamparia, and Devendra Kumar Pandey. 2019. A comprehensive survey of edge prediction in social networks: Techniques, parameters and challenges. *Expert Systems with Applications* 124 (2019), 164–181.
- [86] Hongbin Pei, Bingzhe Wei, Kevin Chen-Chuan Chang, Yu Lei, and Bo Yang. 2020. Geom-GCN: Geometric Graph Convolutional Networks. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=S1e2agrFvS>

- [87] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. 2014. DeepWalk: Online learning of social representations. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*. 701–710.
- [88] Trang Pham, Truyen Tran, Hoa Dam, and Svetha Venkatesh. 2017. Graph classification via deep learning with virtual nodes. *arXiv preprint arXiv:1708.04357* (2017).
- [89] Huyen Trang Phan, Ngoc Thanh Nguyen, and Dosam Hwang. 2023. Fake news detection: A survey of graph neural network methods. *Applied Soft Computing* 139 (2023), 110235.
- [90] Jiezhong Qiu, Yuxiao Dong, Hao Ma, Jian Li, Kuansan Wang, and Jie Tang. 2018. Network embedding as matrix factorization: Unifying DeepWalk, LINE, PTE, and node2vec. In *Proceedings of the eleventh ACM international conference on web search and data mining*. 459–467.
- [91] Patrick Reiser, Marlen Neubert, André Eberhard, Luca Torresi, Chen Zhou, Chen Shao, Houssam Metni, Clint van Hoesel, Henrik Schopmans, Timo Sommer, et al. 2022. Graph neural networks for materials science and chemistry. *Communications Materials* 3, 1 (2022), 93.
- [92] Yazhou Ren, Kangrong Hu, Xinyi Dai, Lili Pan, Steven CH Hoi, and Zenglin Xu. 2019. Semi-supervised deep embedded clustering. *Neurocomputing* 325 (2019), 121–130.
- [93] Leonardo FR Ribeiro, Pedro HP Saverese, and Daniel R Figueiredo. 2017. struc2vec: Learning node representations from structural identity. In *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining*. 385–394.
- [94] Olivier Roy and Martin Vetterli. 2007. The Effective Rank: A Measure of Effective Dimensionality. In *Proceedings of the 15th European Signal Processing Conference (EUSIPCO 2007)*. Poznań, Poland, 606–610.
- [95] Benedek Rozemberczki, Carl Allen, and Rik Sarkar. 2021. Multi-scale attributed node embedding. *Journal of Complex Networks* 9, 2 (2021), cnab014.
- [96] T Konstantin Rusch, Michael M Bronstein, and Siddhartha Mishra. 2023. A survey on oversmoothing in graph neural networks. *arXiv preprint arXiv:2303.10993* (2023).

- [97] Michael T Schaub and Santiago Segarra. 2018. Flow smoothing and denoising: Graph signal processing in the edge-space. In *2018 IEEE Global Conference on Signal and Information Processing (GlobalSIP)*. IEEE, 735–739.
- [98] Michael Schlichtkrull, Thomas N Kipf, Peter Bloem, Rianne Van Den Berg, Ivan Titov, and Max Welling. 2018. Modeling relational data with graph convolutional networks. In *European semantic web conference*. Springer, 593–607.
- [99] K. Shao, Y. Zhang, Y. Wen, Z. Zhang, S. He, and X. Bo. 2022. DTI-HETA: prediction of drug–target interactions based on GCN and GAT on heterogeneous graph. *Briefings in Bioinformatics* 23, 3 (2022), bbac109. doi:10.1093/bib/bbac109
- [100] Yingxia Shao, Hongzheng Li, Xizhi Gu, Hongbo Yin, Yawen Li, Xupeng Miao, Wentao Zhang, Bin Cui, and Lei Chen. 2024. Distributed graph neural network training: A survey. *Comput. Surveys* 56, 8 (2024), 1–39.
- [101] Oleksandr Shchur, Maximilian Mumme, Aleksandar Bojchevski, and Stephan Günnemann. 2018. Pitfalls of graph neural network evaluation. *arXiv preprint arXiv:1811.05868* (2018).
- [102] Natalie R Smith, Paul N Zivich, Leah M Frerichs, James Moody, and Allison E Aiello. 2020. A guide for choosing community detection algorithms in social network studies: The question alignment approach. *American journal of preventive medicine* 59, 4 (2020), 597–605.
- [103] Shyam A Tailor, Felix Opolka, Pietro Lio, and Nicholas Donald Lane. 2021. Do we need anisotropic graph neural networks?. In *International Conference on Learning Representations*.
- [104] Gregory Tauer, Ketan Date, Rakesh Nagi, and Moises Sudit. 2019. An incremental graph-partitioning algorithm for entity resolution. *Information Fusion* 46 (2019), 171–183.
- [105] Jake Topping, Francesco Di Giovanni, Benjamin Paul Chamberlain, Xiaowen Dong, and Michael M. Bronstein. 2022. Understanding over-squashing and bottlenecks on graphs via curvature. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=7UmjRGzp-A>

- [106] Anton Tsitsulin, John Palowitch, Bryan Perozzi, and Emmanuel Müller. 2023. Graph clustering with graph neural networks. *Journal of Machine Learning Research* 24, 127 (2023), 1–21.
- [107] Remco van der Hofstad. 2009. Random Graphs and Complex Networks. <http://www.win.tue.nl/~rhofstad/NotesRGCN.pdf> Lecture notes.
- [108] Laurens Van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-SNE. *Journal of Machine Learning Research* 9, 11 (2008).
- [109] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=rJXMpikCZ>
- [110] Petar Veličković, William Fedus, William L. Hamilton, Pietro Liò, Yoshua Bengio, and R Devon Hjelm. 2019. Deep Graph Infomax. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=rklz9iAcKQ>
- [111] Chenxu Wang, Chongxiao Hao, and Xiaohong Guan. 2020. Hierarchical and overlapping social circle identification in ego networks based on link clustering. *Neurocomputing* 381 (2020), 322–335.
- [112] Shike Wang, Fan Xu, Yunyang Li, Jie Wang, Ke Zhang, Yong Liu, Min Wu, and Jie Zheng. 2021. KG4SL: knowledge graph neural network for synthetic lethality prediction in human cancers. *Bioinformatics* 37, Supplement_1 (2021), i418–i425. doi:10.1093/bioinformatics/btab271
- [113] Xiaoyu Wang, Cailian Chen, Yang Min, Jianping He, Bo Yang, and Yang Zhang. 2018. Efficient metropolitan traffic prediction based on graph recurrent neural network. *arXiv preprint arXiv:1811.00740* (2018).
- [114] Yuyang Wang, Jianren Wang, Zhonglin Cao, and Amir Barati Farimani. 2022. Molecular contrastive learning of representations via graph neural networks. *Nature Machine Intelligence* 4, 3 (2022), 279–287.
- [115] Kanit Wongsuphasawat, Daniel Smilkov, James Wexler, Jimbo Wilson, Dandelion Mane, Doug Fritz, Dilip Krishnan, Fernanda B Vi’egas,

- and Martin Wattenberg. 2017. Visualizing dataflow graphs of deep learning models in TensorFlow. *IEEE Transactions on Visualization and Computer Graphics* 24, 1 (2017), 1–12.
- [116] Felix Wu, Amauri Souza, Tianyi Zhang, Christopher Fifty, Tao Yu, and Kilian Weinberger. 2019. Simplifying graph convolutional networks. In *International Conference on Machine Learning*. PMLR, 6861–6871.
 - [117] Lingfei Wu, Peng Cui, Jian Pei, Liang Zhao, and Xiaojie Guo. 2022. Graph neural networks: foundation, frontiers and applications. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4840–4841.
 - [118] Qitian Wu, Wentao Zhao, Zenan Li, David P Wipf, and Junchi Yan. 2022. NodeFormer: A Scalable Graph Structure Learning Transformer for Node Classification. In *Advances in Neural Information Processing Systems*, Vol. 35. 27387–27401.
 - [119] Shiwen Wu, Fei Sun, Wentao Zhang, Xu Xie, and Bin Cui. 2022. Graph neural networks in recommender systems: a survey. *Comput. Surveys* 55, 5 (2022), 1–37.
 - [120] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and S Yu Philip. 2020. A comprehensive survey on graph neural networks. *IEEE transactions on neural networks and learning systems* 32, 1 (2020), 4–24.
 - [121] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2019. How Powerful are Graph Neural Networks?. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=ryGs6iA5Km>
 - [122] Keyulu Xu, Chengtao Li, Yonglong Tian, Tomohiro Sonobe, Ken-ichi Kawarabayashi, and Stefanie Jegelka. 2018. Representation Learning on Graphs with Jumping Knowledge Networks. In *Proceedings of the 35th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 80)*. PMLR, 5453–5462. <https://proceedings.mlr.press/v80/xu18c.html>
 - [123] Zhiying Xu, Francis Y Yan, Rachee Singh, Justin T Chiu, Alexander M Rush, and Minlan Yu. 2023. Teal: Learning-accelerated optimization

of wan traffic engineering. In *Proceedings of the ACM SIGCOMM 2023 Conference*. 378–393.

- [124] Zhao Yang, René Algesheimer, and Claudio J Tessone. 2016. A comparative analysis of community detection algorithms on artificial networks. *Scientific reports* 6, 1 (2016), 1–18.
- [125] Zhilin Yang, William Cohen, and Ruslan Salakhudinov. 2016. Revisiting semi-supervised learning with graph embeddings. In *International Conference on Machine Learning*. PMLR, 40–48.
- [126] Xueyan Yin, Genze Wu, Jinze Wei, Yanming Shen, Heng Qi, and Baocai Yin. 2020. A comprehensive survey on traffic prediction. *arXiv preprint arXiv:2004.08555* (2020).
- [127] Rex Ying, Dylan Bourgeois, Jiaxuan You, Marinka Zitnik, and Jure Leskovec. 2019. GNNExplainer: Generating Explanations for Graph Neural Networks. In *Proceedings of the 33rd Conference on Neural Information Processing Systems (NeurIPS 2019)*, Vol. 32. Curran Associates, Inc., 9244–9255. https://proceedings.neurips.cc/paper_files/paper/2019/hash/d80b7040b773199015de6d3b4293c8ff-Abstract.html
- [128] Jiaxuan You, Zhitao Ying, and Jure Leskovec. 2020. Design space for graph neural networks. *Advances in Neural Information Processing Systems* 33 (2020).
- [129] Yuning You, Tianlong Chen, Yongduo Sui, Ting Chen, Zhangyang Wang, and Yang Shen. 2020. Graph contrastive learning with augmentations. *Advances in neural information processing systems* 33 (2020), 5812–5823.
- [130] L. Yu, W. Qiu, W. Lin, X. Cheng, X. Xiao, and J. Dai. 2022. HGDTI: predicting drug–target interaction by using information aggregation based on heterogeneous graph neural network. *BMC Bioinformatics* 23, 1 (2022), 126. doi:10.1186/s12859-022-04655-5
- [131] Jiani Zhang, Xingjian Shi, Junyuan Xie, Hao Ma, Irwin King, and Dit Yan Yeung. 2018. GaAN: Gated Attention Networks for Learning on Large and Spatiotemporal Graphs. In *34th Conference on Uncertainty in Artificial Intelligence 2018, UAI 2018*.

- [132] Muhan Zhang and Yixin Chen. 2018. Link prediction based on graph neural networks. *Advances in Neural Information Processing Systems* 31 (2018), 5165–5175.
- [133] Muhan Zhang, Zhicheng Cui, Marion Neumann, and Yixin Chen. 2018. An end-to-end deep learning architecture for graph classification. In *Thirty-Second AAAI Conference on Artificial Intelligence*.
- [134] Si Zhang, Hanghang Tong, Jiejun Xu, and Ross Maciejewski. 2019. Graph convolutional networks: a comprehensive review. *Computational Social Networks* 6, 1 (2019), 1–23.
- [135] Tong Zhang, Ming-Yu Xie, Yan Li, and Shao-Wu Zhang. 2023. A Novel Heterophilic Graph Diffusion Convolutional Network for Identifying Cancer Driver Genes. *Briefings in Bioinformatics* 24, 3 (2023), bbad137. doi:10.1093/bib/bbad137
- [136] Ziwei Zhang, Peng Cui, and Wenwu Zhu. 2020. Deep learning on graphs: A survey. *IEEE Transactions on Knowledge and Data Engineering* (2020).
- [137] Zaixi Zhang, Qi Liu, Hao Wang, Chengqiang Lu, and Cheekong Lee. 2022. ProtGNN: Towards Self-Explaining Graph Neural Networks. In *Proceedings of the 36th AAAI Conference on Artificial Intelligence (AAAI-22)*, Vol. 36. Association for the Advancement of Artificial Intelligence, 9127–9135. <https://ojs.aaai.org/index.php/AAAI/article/view/20898>
- [138] Lingxiao Zhao and Leman Akoglu. 2020. PairNorm: Tackling Over-smoothing in GNNs. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=rkecl1rtwB>
- [139] Chuanpan Zheng, Xiaoliang Fan, Cheng Wang, and Jianzhong Qi. 2020. GMAN: A graph multi-attention network for traffic prediction. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 34. 1234–1241.
- [140] Xin Zheng, Yi Wang, Yixin Liu, Ming Li, Miao Zhang, Di Jin, Philip S Yu, and Shirui Pan. 2022. Graph neural networks for graphs with heterophily: A survey. *arXiv preprint arXiv:2202.07082* (2022).
- [141] Jie Zhou, Ganqu Cui, Shengding Hu, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, Lifeng Wang, Changcheng Li, and Maosong Sun. 2020.

- Graph neural networks: A review of methods and applications. *AI Open* 1 (2020), 57–81.
- [142] Kuangqi Zhou, Yanfei Dong, Kaixin Wang, Wee Sun Lee, Bryan Hooi, Huan Xu, and Jiashi Feng. 2021. Understanding and resolving performance degradation in deep graph convolutional networks. In *Proceedings of the 30th ACM international conference on information & knowledge management*. 2728–2737.
 - [143] Kaixiong Zhou, Xiao Huang, Yuening Li, Daochen Zha, Rui Chen, and Xia Hu. 2020. Towards deeper graph neural networks with differentiable group normalization. *Advances in neural information processing systems* 33 (2020), 4917–4928.
 - [144] Yu Zhou, Haixia Zheng, Xin Huang, Shufeng Hao, Dengao Li, and Jumin Zhao. 2022. Graph Neural Networks: Taxonomy, Advances, and Trends. *ACM Transactions on Intelligent Systems and Technology (TIST)* 13, 1 (2022), 1–54.
 - [145] Jiong Zhu, Ryan A Rossi, Anup Rao, Tung Mai, Nedim Lipka, Nesreen K Ahmed, and Danai Koutra. 2021. Graph neural networks with heterophily. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 35. 11168–11176.
 - [146] Jiong Zhu, Yujun Yan, Lingxiao Zhao, Mark Heimann, Leman Akoglu, and Danai Koutra. 2020. Beyond homophily in graph neural networks: Current limitations and effective designs. *Advances in neural information processing systems* 33 (2020), 7793–7804.
 - [147] Xiaojin Zhu and Zoubin Ghahramani. 2002. *Learning from Labeled and Unlabeled Data with Label Propagation*. Technical Report CMU-CALD-02-107. Carnegie Mellon University.
 - [148] Zhijian Zhuo, Yifei Wang, Jinwen Ma, and Yisen Wang. 2023. Towards a Unified Theoretical Understanding of Non-contrastive Learning via Rank Differential Mechanism. In *Proceedings of the International Conference on Learning Representations (ICLR)*. arXiv:2303.02387.

S Supplementary Material

S.1 Formal definitions of metrics

For reference, we have an attributed graph $\mathbf{G} = (\mathcal{N}, A, (x_i^T)_{i \in \mathcal{N}})$.

S.1.1 Fisher SNR with eigenvalue-floor shrinkage

Suppose that the nodes $\mathcal{N}$ have C classes. Let $\{(x_i, y_i)\}_{i=1}^N$ with $x_i \in \mathbb{R}^d$, $y_i \in \{1, \dots, C\}$. For class c , let $S_c = \{i : y_i = c\}$, $n_c = |S_c|$, and

$$\mu_c = \frac{1}{n_c} \sum_{i \in S_c} x_i, \quad \bar{\mu} = \frac{1}{C} \sum_{c=1}^C \mu_c.$$

The pooled within-class covariance is

$$\Sigma_w = \frac{1}{\sum_{c=1}^C (n_c - 1)} \sum_{c=1}^C \sum_{i \in S_c} (x_i - \mu_c)(x_i - \mu_c)^\top.$$

Let $\Sigma_w = U\Lambda U^\top$ be an eigendecomposition with

$$\Lambda = \text{diag}(\lambda_1, \dots, \lambda_d), \quad \lambda_i \geq 0.$$

To avoid spurious blow-ups due purely to small-sample noise, we apply a mild eigenvalue floor to the pooled within-class covariance Σ_w . Define $\bar{\lambda} = \frac{1}{d} \text{tr}(\Sigma_w)$. For a small $\alpha \in (0, 1)$, define the clamped spectrum

$$\Lambda^{(\alpha)} = \text{diag}(\max\{\lambda_1, \alpha\bar{\lambda}\}, \dots, \max\{\lambda_d, \alpha\bar{\lambda}\}),$$

and the shrunk covariance

$$\Sigma_w^{(\alpha)} = U\Lambda^{(\alpha)}U^\top.$$

This guarantees that the shrunk covariance $\Sigma_w^{(\alpha)}$ is well-conditioned.

The stable Fisher signal-to-noise ratio is

$$\text{SNR} = \frac{1}{C^2} \sum_{1 \leq b, c \leq C} (\mu_b - \mu_c)^\top (\Sigma_w^{(\alpha)})^{-1} (\mu_b - \mu_c).$$

S.1.2 Within-class Effective Rank

Let Σ_w be defined as in Section S.1.1. Let $\lambda_1, \dots, \lambda_d \geq 0$ be the eigenvalues of Σ_w , and define normalized eigenvalue weights

$$p_i = \frac{\lambda_i}{\sum_{j=1}^d \lambda_j}, \quad i = 1, \dots, d.$$

Their Shannon entropy (with natural logarithms) is

$$H(p) = - \sum_{i=1}^d p_i \log p_i.$$

The *Within-Class Effective Rank* is defined as the entropy-based effective rank of Σ_w :

$$r_{\text{WC}}(\Sigma_w) = \exp(H(p)) = \exp\left(- \sum_{i=1}^d p_i \log p_i\right).$$

This quantity satisfies $1 \leq r_{\text{WC}}(\Sigma_w) \leq d$ and can be interpreted as the effective number of independent directions of within-class variation.

S.1.3 Nearest-Centroid 5th-Percentile Margin (NCMq0.05)

Let $T, S \subseteq \{1, \dots, N\}$ denote the index sets of training and test nodes, respectively. For each class $c \in \{1, \dots, C\}$, define the class-specific training set and size

$$T_c = \{i \in T : y_i = c\}, \quad n_c^{\text{tr}} = |T_c|,$$

and the corresponding training centroid

$$\mu_c^{\text{tr}} = \frac{1}{n_c^{\text{tr}}} \sum_{i \in T_c} x_i.$$

Let $\Sigma_{w,\text{tr}}$ be the pooled within-class covariance computed on the training set $\{x_i\}_{i \in T}$ analogously to Σ_w in Section S.1.1, and let $\Sigma_{w,\text{tr}}^{(\alpha)}$ denote its eigenvalue-floor shrunk version (as in the definition of $\Sigma_w^{(\alpha)}$). We choose a whitening transform $W \in \mathbb{R}^{d \times d}$ such that

$$W \Sigma_{w,\text{tr}}^{(\alpha)} W^\top = I_d.$$

We then define the whitened embeddings and whitened training centroids by

$$z_i = W x_i, \quad \tilde{\mu}_c = W \mu_c^{\text{tr}}, \quad c = 1, \dots, C.$$

In this whitened space, the nearest-centroid classifier predicts

$$\hat{y}_i = \arg \min_{c \in \{1, \dots, C\}} \|z_i - \tilde{\mu}_c\|_2^2.$$

For a test node $i \in S$ with true class $c = y_i$ and any competing class $j \neq c$, the signed Euclidean distance from z_i to the decision hyperplane between classes c and j (positive on the side of class c) is

$$m_i(c, j) = \frac{(\tilde{\mu}_c - \tilde{\mu}_j)^\top z_i - \frac{1}{2}(\|\tilde{\mu}_c\|_2^2 - \|\tilde{\mu}_j\|_2^2)}{\|\tilde{\mu}_c - \tilde{\mu}_j\|_2}.$$

The nearest-centroid margin of node i is then

$$m_i = \min_{j \neq c} m_i(c, j), \quad c = y_i.$$

Let $\{m_i : i \in S\}$ be the collection of margins for all test nodes. We define the *NCMq0.05* metric as the 5th-percentile nearest-centroid margin,

$$\text{NCMq0.05} = \text{Quantile}_{0.05}(\{m_i : i \in S\}).$$

Higher values of *NCMq0.05* indicate that the hardest 5% of nodes remain farther from any decision boundary, whereas lower (more negative) values indicate that a larger fraction of nodes lie close to or across a decision boundary. See Figure 12, which shows a trend of decreasing margins as the networks get deeper.

S.1.4 Message Squeeze per Bridge Edge

Let $\mathcal{E}$ be the set of edges in $\mathbf{G}$. Each node $n \in \mathcal{N}$ has a feature vector $x_n \in \mathbb{R}^d$, and a trained graph neural network (GNN) produces for each node $n \in \mathcal{N}$ a scalar output $\phi(n)$ (for example, the logit of the predicted class at n).

1. Distance, ball, boundary, and far set. Let $\text{dist}(u, n)$ denote the shortest-path distance between nodes u and n in $\mathcal{N}$. For a fixed node $n \in \mathcal{N}$ and radius $r < 0$, define the r -hop ball around n by

$$B_r(n) = \{u \in \mathcal{N} : \text{dist}_G(u, n) \leq r\}. \quad (62)$$

The boundary (set of *bridge edges*) of this ball is

$$\partial B_r(n) = \{(u, v) \in E : u \in B_r(n), v \notin B_r(n) \text{ or vice versa}\}. \quad (63)$$

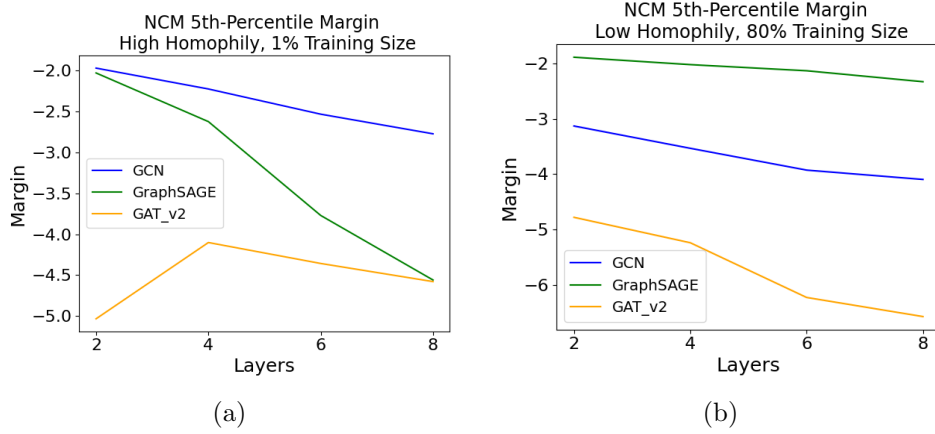


Figure 12: NCM 5th-percentile margin of node embeddings as a function of GNN depth for three architectures. The NCM 5th-percentile margin is the 5th percentile of nearest-centroid margins in a whitened embedding space; lower values indicate that a larger fraction of nodes lie close to decision boundaries. Left: high homophily graphs with 1% of nodes labeled for training. Right: low homophily graphs with 80% of nodes labeled for training. In both regimes, margins decrease with depth beyond four layers for all models, indicating that an increasing fraction of nodes lie near decision boundaries as the networks become deeper.

The corresponding *far set* is

$$F_r(n) = \{u \in \mathcal{N} : \text{dist}(u, n) > r\}. \quad (64)$$

2. Node-to-node influence. For a node $u \in \mathcal{N}$, define the *influence* of u on n via the gradient of $\phi(n)$ with respect to x_u . Let $g(u \rightarrow n) \in \mathbb{R}^d$ be this gradient:

$$g(u \rightarrow n) = \frac{\partial \phi(n)}{\partial x_u} \in \mathbb{R}^d. \quad (65)$$

Let $\sigma \in \mathbb{R}^d$ be a vector of feature-wise standard deviations (e.g., computed over all node features in the dataset), and let $\sigma_k > 0$ for $k = 1, \dots, d$. Define the standardized gradient $\tilde{g}(u \rightarrow n)$ by

$$\tilde{g}_k(u \rightarrow n) = \frac{1}{\sigma_k} g_k(u \rightarrow n), \quad k = 1, \dots, d. \quad (66)$$

Let $d_{\text{eff}} \in \mathbb{N}$ be the number of nonzero entries of $\tilde{g}(u \rightarrow n)$ (or simply d if

all features are used). Then the *node-to-node influence* is

$$J(u \rightarrow n) = \frac{1}{\sqrt{d_{\text{eff}}}} \|\tilde{g}(u \rightarrow n)\|_2. \quad (67)$$

3. Message Squeeze per Bridge Edge. For a fixed node n and radius r , the *Message Squeeze per Bridge Edge* (MSBE) at n is defined as

$$\text{MSBE}_r(n) = \frac{1}{|\partial B_r(n)| + \varepsilon} \sum_{u \in F_r(n)} J(u \rightarrow n), \quad (68)$$

where $|\partial B_r(n)|$ is the number of bridge edges of the ball $B_r(n)$ and $\varepsilon > 0$ is a small constant to avoid division by zero.

S.1.5 Change in Message Squeeze per Bridge Edge

For a fixed node $n \in \mathcal{N}$ and radius $r > 0$, we consider adding a small set of *shortcut edges* from n into its far set $F_r(n)$. Given an integer $M \geq 1$, we select a set of M far nodes $S_r(n) \subseteq F_r(n)$ by taking the M nodes in $F_r(n)$ with the largest influence $J(u \rightarrow n)$. These define shortcut edges

$$\mathcal{E}^{\text{short}}(n) = \{ (n, u) : u \in S_r(n) \}, \quad (69)$$

and a corresponding rewired edge set

$$\mathcal{E}^+(n) = \mathcal{E} \cup \mathcal{E}^{\text{short}}(n). \quad (70)$$

We write $J^+(u \rightarrow n)$ for the node-to-node influence defined as above, but computed on the rewired graph (with edges $\mathcal{E}^+(n)$) with the same trained GNN parameters. Then the *change in Message Squeeze per Bridge Edge* at n is then

$$\Delta \text{MSBE}_r(n) = \frac{\sum_{u \in F_r(n)} J^+(u \rightarrow n) - \sum_{u \in F_r(n)} J(u \rightarrow n)}{|\partial B_r(n)| + \varepsilon}. \quad (71)$$

S.2 Architectures Selected by Simple Tuning

The following tables report statistics on the tuned GNN designs from Section 5.4.1.

Table 12: The mean value and p -value of the number of layers and the learning rates of the hyperparameter tuned models. The hyperparameters were tuned for each model on the low and high homophily dataset collections with 1% and 80% of node labels for training. For each hyperparameter, the p -value is for the null hypothesis that the selections over the collection of datasets are drawn from a random distribution.

Statistics of Design Parameters of Tuned GNNs								
80% Training	Pre Layers		MP Layers		Post Layers		LR	
Low Homophily	mean	p -value	mean	p -value	mean	p -value	mean	p -value
GCN	2.17	0.23	3.67	0.21	3.0	0.0	0.024	0.0
GATv2	2.0	0.41	3.83	0.27	2.83	0.0	0.012	0.17
GraphSAGE	1.83	0.4	6.5	0.01	2.17	0.23	0.011	0.34
1% Training								
High Homophily	mean	p -value	mean	p -value	mean	p -value	mean	p -value
GCN	1.57	0.12	6.86	0.0	1.43	0.05	0.010	0.37
GATv2	1.71	0.25	5.86	0.05	1.43	0.05	0.010	0.46
GraphSAGE	2.29	0.13	7.71	0.0	1.71	0.25	0.0075	0.02

Table 13: The most common and % occurring values of the skip connections and aggregation functions on the low and high homophily dataset collections, with 1% and 80% of node labels for training. The hyperparameters for the algorithms have been tuned to each dataset. The % occurring field says how often the most common selection occurred.

Statistics of Design Parameters of Tuned GNNs				
80% Training	Skip Connections		Aggregation	
Low Homophily	most common	% occurring	most common	% occurring
GCN	skip concat	66.67	max	50.0
GATv2	skip concat	100.0	add	50.0
GraphSAGE	skip sum	66.67	add	66.67
1% Training				
High Homophily	most common	% occurring	most common	% occurring
GCN	skip sum	71.43	max	71.43
GATv2	skip sum	100.0	max	57.14
GraphSAGE	none	71.43	add	42.86

Table 14: The node classification accuracy of default designs (see Table 4).

Node Classification Accuracy: 80% Training and 16 Hidden Dimensions			
Dataset	GCN	GATv2	GraphSAGE
Photo	89.48 ± 3.47	93.55 ± 0.37	77.95 ± 8.99
DBLP	85.16 ± 0.29	85.11 ± 0.32	84.84 ± 0.30
Cora	88.28 ± 0.74	85.71 ± 0.79	88.25 ± 0.66
CoauthorCS	93.22 ± 0.21	91.24 ± 0.29	94.26 ± 0.21
PubMed	87.21 ± 0.24	86.99 ± 0.26	87.89 ± 0.38
Computers	77.34 ± 5.09	89.88 ± 0.39	77.45 ± 5.16
CiteSeer	76.46 ± 0.93	76.19 ± 0.88	75.77 ± 1.03
Crocodile	61.66 ± 0.96	68.26 ± 0.69	73.26 ± 0.44
Chameleon	46.90 ± 1.17	55.34 ± 1.39	64.07 ± 1.30
Squirrel	29.53 ± 1.07	37.68 ± 0.73	45.24 ± 0.70
Cctor	27.66 ± 0.68	28.89 ± 0.65	35.08 ± 0.64
Wisconsin	32.00 ± 3.30	40.00 ± 2.80	66.31 ± 4.05
Cornell	28.42 ± 2.80	39.16 ± 3.25	56.21 ± 4.11

S.3 Tables of baseline GNN results on individual datasets

All tables report results with 95% confidence intervals based on 25 runs.

Table 15: The node classification accuracy of default designs (see Table 4).

Node Classification Accuracy: 1% Training and 16 Hidden Dimensions			
Dataset	GCN	GATv2	GraphSAGE
Photo	78.35 ± 7.05	78.71 ± 1.81	65.22 ± 5.18
DBLP	74.80 ± 0.69	74.63 ± 0.89	72.50 ± 0.96
Cora	60.55 ± 2.78	54.89 ± 2.78	55.93 ± 1.52
CoauthorCS	89.94 ± 0.22	77.13 ± 0.79	86.10 ± 0.36
PubMed	81.16 ± 0.37	79.14 ± 0.46	78.86 ± 0.33
Computers	74.83 ± 4.76	72.64 ± 1.63	54.20 ± 4.22
CiteSeer	48.38 ± 1.70	48.74 ± 2.26	43.70 ± 1.88
Crocodile	46.44 ± 1.58	45.38 ± 1.67	52.91 ± 1.30
Chameleon	30.58 ± 1.59	30.56 ± 1.35	31.47 ± 1.77
Squirrel	22.16 ± 0.57	22.39 ± 0.66	26.93 ± 1.08
Cctor	24.44 ± 0.38	23.98 ± 0.49	24.48 ± 0.62
Wisconsin	34.03 ± 5.74	38.60 ± 4.18	40.22 ± 4.83
Cornell	28.26 ± 5.58	24.65 ± 5.56	29.96 ± 5.05

Table 16: The node classification accuracy of default designs (see Table 5).

Node Classification Accuracy: 80% Training and 64 Hidden Dimensions			
Dataset	GCN	GATv2	GraphSAGE
Photo	93.69 ± 0.26	93.54 ± 0.36	94.24 ± 0.54
DBLP	85.69 ± 0.31	85.03 ± 0.26	84.72 ± 0.36
Cora	88.69 ± 0.70	86.10 ± 0.78	88.59 ± 0.68
CoauthorCS	93.48 ± 0.20	92.04 ± 0.33	94.69 ± 0.16
PubMed	87.64 ± 0.29	87.27 ± 0.39	88.23 ± 0.36
Computers	88.96 ± 0.42	90.27 ± 0.31	88.83 ± 0.75
CiteSeer	77.09 ± 0.82	75.68 ± 0.96	75.95 ± 0.68
Crocodile	63.46 ± 0.75	68.93 ± 0.50	73.05 ± 0.37
Chameleon	47.48 ± 1.72	56.16 ± 1.23	64.21 ± 0.97
Squirrel	30.20 ± 0.72	38.50 ± 0.87	44.85 ± 0.77
Cctor	28.59 ± 0.57	28.37 ± 0.85	34.87 ± 0.65
Wisconsin	36.62 ± 2.83	38.92 ± 3.82	68.62 ± 3.29
Cornell	28.63 ± 4.13	40.00 ± 4.25	61.89 ± 4.01

Table 17: The node classification accuracy of default designs (see Table 5).

Node Classification Accuracy: 1% Training and 64 Hidden Dimensions			
Dataset	GCN	GATv2	GraphSAGE
Photo	89.13 ± 0.69	81.97 ± 1.02	80.10 ± 1.56
DBLP	76.01 ± 0.50	75.08 ± 0.94	73.99 ± 0.65
Cora	62.96 ± 2.78	61.57 ± 2.32	58.48 ± 2.77
CoauthorCS	89.81 ± 0.26	80.42 ± 0.75	86.80 ± 0.55
PubMed	80.98 ± 0.40	79.66 ± 0.42	79.14 ± 0.38
Computers	83.09 ± 0.49	76.20 ± 0.81	71.25 ± 3.31
CiteSeer	51.01 ± 1.87	49.99 ± 2.21	50.67 ± 2.07
Crocodile	46.40 ± 1.59	47.51 ± 1.30	53.96 ± 0.65
Chameleon	31.84 ± 1.19	30.03 ± 1.80	33.39 ± 1.38
Squirrel	22.66 ± 0.62	22.84 ± 0.57	27.80 ± 0.82
Cctor	23.26 ± 0.80	23.91 ± 0.66	24.78 ± 0.54
Wisconsin	38.32 ± 4.71	33.56 ± 4.93	41.40 ± 5.31
Cornell	29.26 ± 6.03	25.09 ± 5.55	33.26 ± 6.12

Table 18: The node classification accuracy of default designs (see Table 5).

Node Classification Accuracy: 80% Training and 128 Hidden Dimensions			
Dataset	GCN	GATv2	GraphSAGE
Photo	93.87 ± 0.32	93.21 ± 0.29	95.24 ± 0.37
DBLP	85.69 ± 0.34	85.33 ± 0.26	84.89 ± 0.22
Cora	88.81 ± 0.65	85.97 ± 0.72	88.29 ± 0.70
CoauthorCS	93.49 ± 0.20	92.34 ± 0.29	94.73 ± 0.20
PubMed	88.19 ± 0.34	87.31 ± 0.29	88.65 ± 0.29
Computers	90.23 ± 0.31	90.27 ± 0.34	89.96 ± 0.51
CiteSeer	76.91 ± 0.88	76.19 ± 0.93	76.80 ± 0.94
Crocodile	63.13 ± 0.53	69.28 ± 0.67	73.42 ± 0.60
Chameleon	47.06 ± 1.88	54.46 ± 1.33	64.14 ± 1.34
Squirrel	29.16 ± 0.85	39.45 ± 0.88	45.02 ± 0.73
Cctor	28.42 ± 0.66	29.06 ± 0.48	35.15 ± 0.54
Wisconsin	36.92 ± 3.25	40.62 ± 3.28	68.00 ± 2.95
Cornell	31.37 ± 3.48	40.42 ± 4.20	57.89 ± 3.40

Table 19: The node classification accuracy of default designs (see Table 5).

Node Classification Accuracy: 1% Training and 128 Hidden Dimensions			
Dataset	GCN	GATv2	GraphSAGE
Photo	89.57 ± 0.63	81.27 ± 0.94	80.39 ± 1.61
DBLP	76.43 ± 0.69	75.21 ± 0.71	74.24 ± 0.54
Cora	64.08 ± 2.31	60.72 ± 2.97	58.55 ± 2.20
CoauthorCS	89.76 ± 0.24	81.40 ± 0.84	86.25 ± 0.50
PubMed	81.06 ± 0.33	79.19 ± 0.36	79.54 ± 0.28
Computers	83.53 ± 0.50	75.27 ± 1.04	72.91 ± 2.27
CiteSeer	50.80 ± 1.88	51.95 ± 1.86	51.30 ± 2.08
Crocodile	46.90 ± 1.72	47.44 ± 2.25	54.55 ± 0.85
Chameleon	31.24 ± 1.63	30.40 ± 1.36	34.75 ± 1.59
Squirrel	23.46 ± 0.51	22.36 ± 0.67	28.84 ± 0.77
Cctor	24.11 ± 0.64	23.71 ± 0.62	25.15 ± 0.45
Wisconsin	33.30 ± 5.34	36.13 ± 5.30	34.29 ± 5.57
Cornell	28.61 ± 5.98	31.30 ± 5.87	36.30 ± 4.84

Table 20: The node classification accuracy of default designs (see Table 5).

Node Classification Accuracy: 80% Training and 256 Hidden Dimensions			
Dataset	GCN	GATv2	GraphSAGE
Photo	93.77 ± 0.24	93.46 ± 0.37	94.94 ± 0.30
DBLP	85.33 ± 0.33	84.78 ± 0.38	84.78 ± 0.36
Cora	88.13 ± 1.00	86.69 ± 0.79	88.35 ± 0.91
CoauthorCS	93.37 ± 0.28	92.34 ± 0.33	94.57 ± 0.20
PubMed	88.31 ± 0.30	87.10 ± 0.26	88.52 ± 0.21
Computers	90.33 ± 0.36	90.06 ± 0.35	90.56 ± 0.32
CiteSeer	76.59 ± 0.66	75.02 ± 1.00	76.40 ± 0.96
Crocodile	64.04 ± 0.66	69.49 ± 0.54	73.29 ± 0.51
Chameleon	47.13 ± 1.35	56.14 ± 1.35	63.44 ± 1.21
Squirrel	29.07 ± 0.62	38.93 ± 0.85	45.93 ± 0.76
Cctor	28.13 ± 0.56	28.71 ± 0.70	35.62 ± 0.60
Wisconsin	34.92 ± 3.49	38.46 ± 3.25	68.77 ± 2.77
Cornell	30.11 ± 3.33	39.58 ± 5.06	56.84 ± 3.20

Table 21: The node classification accuracy of default designs (see Table 5).

Node Classification Accuracy: 1% Training and 256 Hidden Dimensions			
Dataset	GCN	GATv2	GraphSAGE
Photo	89.08 $\pm$ 0.69	78.98 $\pm$ 1.62	81.93 $\pm$ 1.45
DBLP	76.65 $\pm$ 0.61	75.29 $\pm$ 0.93	74.45 $\pm$ 0.52
Cora	66.21 $\pm$ 2.31	62.17 $\pm$ 2.59	61.51 $\pm$ 1.83
CoauthorCS	89.64 $\pm$ 0.23	81.75 $\pm$ 1.28	84.78 $\pm$ 0.98
PubMed	81.21 $\pm$ 0.33	79.07 $\pm$ 0.44	79.39 $\pm$ 0.37
Computers	83.59 $\pm$ 0.56	72.81 $\pm$ 0.88	74.99 $\pm$ 1.95
CiteSeer	52.50 $\pm$ 2.28	50.94 $\pm$ 2.04	52.20 $\pm$ 1.44
Crocodile	47.64 $\pm$ 1.37	46.73 $\pm$ 1.63	54.07 $\pm$ 0.73
Chameleon	30.94 $\pm$ 1.69	32.71 $\pm$ 1.38	33.91 $\pm$ 1.35
Squirrel	23.23 $\pm$ 0.53	22.62 $\pm$ 0.54	28.52 $\pm$ 0.83
Cctor	24.67 $\pm$ 0.42	24.42 $\pm$ 0.61	25.73 $\pm$ 0.55
Wisconsin	36.22 $\pm$ 5.71	33.43 $\pm$ 5.22	40.16 $\pm$ 5.84
Cornell	27.91 $\pm$ 5.30	25.09 $\pm$ 6.26	24.39 $\pm$ 5.77

Table 22: The node classification accuracy of the tuned designs with 80% of nodes labeled for training (see Table 8).

Node Classification Accuracy with 80% Training			
Dataset	GCN _{tuned}	GraphSAGE _{tuned}	GATv2 _{tuned}
Photo	95.62 $\pm$ 0.18	95.68 $\pm$ 0.16	95.65 $\pm$ 0.18
DBLP	84.86 $\pm$ 0.18	84.59 $\pm$ 0.22	84.89 $\pm$ 0.17
Cora	88.00 $\pm$ 0.49	87.81 $\pm$ 0.47	88.26 $\pm$ 0.46
CoauthorCS	95.36 $\pm$ 0.12	95.17 $\pm$ 0.14	95.17 $\pm$ 0.13
PubMed	90.04 $\pm$ 0.22	89.89 $\pm$ 0.16	89.99 $\pm$ 0.21
Computers	91.95 $\pm$ 0.16	91.56 $\pm$ 0.18	91.95 $\pm$ 0.15
CiteSeer	75.35 $\pm$ 0.62	74.88 $\pm$ 0.49	75.41 $\pm$ 0.48
Crocodile	70.31 $\pm$ 0.36	70.31 $\pm$ 0.34	70.74 $\pm$ 0.34
Chameleon	61.20 $\pm$ 0.80	61.01 $\pm$ 0.78	60.66 $\pm$ 0.82
Squirrel	40.29 $\pm$ 0.62	42.23 $\pm$ 0.59	40.79 $\pm$ 0.68
Actor	35.19 $\pm$ 0.47	35.68 $\pm$ 0.36	35.26 $\pm$ 0.46
Wisconsin	87.37 $\pm$ 1.66	89.73 $\pm$ 1.80	87.45 $\pm$ 2.36
Cornell	79.57 $\pm$ 2.29	81.95 $\pm$ 2.00	76.76 $\pm$ 1.99

Table 23: The node classification accuracy of tuned designs with 1% of nodes labeled for training (see Table 8).

Node Classification Accuracy with 1% Training			
Dataset	GCN _{tuned}	GraphSAGE _{tuned}	GATv2 _{tuned}
Photo	88.71 $\pm$ 0.68	85.98 $\pm$ 0.87	87.96 $\pm$ 0.78
DBLP	75.51 $\pm$ 0.60	73.75 $\pm$ 0.74	75.64 $\pm$ 0.60
Cora	66.62 $\pm$ 1.52	66.65 $\pm$ 2.15	63.53 $\pm$ 2.16
CoauthorCS	90.60 $\pm$ 0.43	89.59 $\pm$ 0.60	90.52 $\pm$ 0.22
PubMed	79.71 $\pm$ 0.40	78.06 $\pm$ 0.43	79.50 $\pm$ 0.45
Computers	82.54 $\pm$ 0.50	82.08 $\pm$ 0.80	83.51 $\pm$ 0.51
CiteSeer	58.01 $\pm$ 1.50	55.32 $\pm$ 1.94	56.32 $\pm$ 1.45
Crocodile	56.12 $\pm$ 0.78	57.02 $\pm$ 0.76	55.54 $\pm$ 0.60
Chameleon	33.46 $\pm$ 1.39	33.42 $\pm$ 1.22	33.52 $\pm$ 1.38
Squirrel	25.88 $\pm$ 0.59	27.70 $\pm$ 0.70	25.80 $\pm$ 0.47
Actor	25.65 $\pm$ 0.35	25.82 $\pm$ 0.51	25.66 $\pm$ 0.45
Wisconsin	27.93 $\pm$ 0.08	27.69 $\pm$ 0.42	27.69 $\pm$ 0.45
Cornell	17.25 $\pm$ 1.23	18.90 $\pm$ 1.52	17.52 $\pm$ 1.30

Table 24: The node classification accuracy of RevGNNs with 80% of node labels for training on low homophily graphs (see Table 10).

Node Classification Accuracy of RevGNNs			
Dataset	RevGCN	RevSAGE	RevGATv2
Crocodile	69.01 $\pm$ 0.55	73.10 $\pm$ 0.53	71.36 $\pm$ 0.54
Chameleon	54.60 $\pm$ 1.18	64.10 $\pm$ 1.36	60.00 $\pm$ 1.49
Squirrel	36.25 $\pm$ 0.90	46.34 $\pm$ 1.07	42.63 $\pm$ 0.80
Actor	33.29 $\pm$ 0.61	36.91 $\pm$ 0.73	36.22 $\pm$ 0.67
Wisconsin	61.38 $\pm$ 3.50	71.08 $\pm$ 2.73	63.85 $\pm$ 3.28
Cornell	51.16 $\pm$ 4.06	66.11 $\pm$ 2.80	47.79 $\pm$ 2.91

Table 25: The node classification accuracy of RevGNNs with 1% of node labels for training on high homophily graphs (see Table 10).

Node Classification Accuracy of RevGNNs			
Dataset	RevGCN	RevSAGE	RevGATv2
Photo	88.89 $\pm$ 0.86	87.12 $\pm$ 0.81	90.21 $\pm$ 0.54
DBLP	77.73 $\pm$ 0.69	76.21 $\pm$ 0.72	77.71 $\pm$ 0.68
Cora	67.37 $\pm$ 1.63	62.41 $\pm$ 2.03	67.96 $\pm$ 1.94
CoauthorCS	91.59 $\pm$ 0.34	90.72 $\pm$ 0.29	90.86 $\pm$ 0.34
PubMed	81.83 $\pm$ 0.39	79.98 $\pm$ 0.36	81.02 $\pm$ 0.45
Computers	83.19 $\pm$ 0.59	79.80 $\pm$ 0.66	83.42 $\pm$ 0.46
CiteSeer	54.41 $\pm$ 1.74	54.87 $\pm$ 1.53	54.73 $\pm$ 1.39